Lecture 10 - Complexity Reduction in Model Predictive Control

Dinesh Krishnamoorthy

Department of Engineering Cybernetics, NTNU

Fall 2026

Previous lecture: Why hybrid / mixed-integer MPC?

Standard MPC assumes continuous inputs $u \in \mathcal{U} \subseteq \mathbb{R}^m$. But many real decisions are discrete:

- on/off equipment: pumps, compressors, heaters, valves;
- how many units to run (parallel chillers, boilers);
- mode selection, switches (e.g., gear changes, PWA models);
- Disjoint constraint sets;
- Logics and rules, e.g., "if temperature $>$ limit, then ..." type conditions.

Hybrid MPC: optimal control of systems mixing continuous dynamics with discrete / logic decisions $\Rightarrow$ mixed-integer optimization.

Mixed integer MPCs are complex to be solved in real-time for many applications?

Solving mixed-integer problems is NP-hard

What does "NP-hard" mean? (informally)

- P: problems solvable in *polynomial time* — "efficient" (e.g. a convex QP).
- NP: problems where a *proposed* solution can be *checked* quickly, even if finding one is hard. *Easy to verify, hard to solve* (like a filled-in Sudoku).
- NP-hard: at least as hard as the hardest problems in NP. No known algorithm solves them in polynomial time; worst-case effort grows exponentially with problem size.

Consequence for MPC: solve time has *no polynomial guarantee* — a real concern when we must solve within a fixed sampling interval.

How MIQPs are solved: branch & bound (high level)

We never enumerate all 2^m combinations. Branch & bound explores a tree of subproblems and prunes most of it.

- **Relax.** Drop integrality ($\delta \in \{0,1\} \to \delta \in [0,1]$) and solve the convex QP relaxation. Its optimum is a lower bound on the true cost.
- **Branch.** If a δ comes out fractional (say 0.4), split into two subproblems: fix $\delta = 0$ and fix $\delta = 1$. Recurse.
- **Bound.** The best integer-feasible solution found so far is the incumbent (an upper bound).
- **Prune.** If a node's relaxation is already worse than the incumbent, discard that *entire* subtree — no need to explore it.

Pruning is the whole trick: it lets us find the *global* optimum while touching only a small fraction of the 2^m leaves — but the worst case is still exponential.

Solving MI-MPC: the branch-and-bound idea

At each sampling time we must solve a mixed-integer QP (MIQP)

$$\min_{u,\,\delta} \; J(u,\delta) \quad \text{s.t.} \quad \text{dynamics, constraints,} \qquad \delta_k \in \{0,1\}^m,\; k = 0,\dots,N-1$$

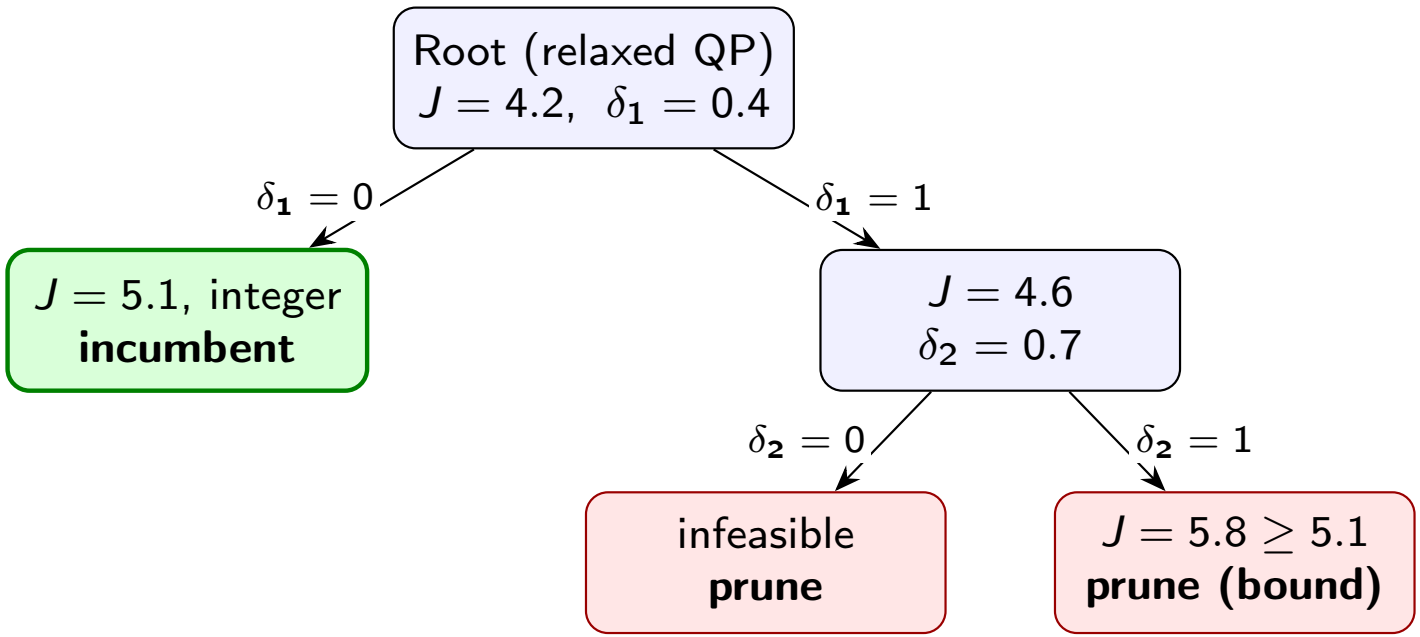
Key trick – relaxation: replace $\delta_k \in \{0,1\}$ by $\delta_k \in [0,1]$ $\Rightarrow$ a convex QP, cheap to solve.

- The relaxed feasible set is *larger* $\Rightarrow$ $J^\star_{\text{relaxed}} \leq J^\star_{\text{MIQP}}$: a lower bound.
- Any integer-feasible solution gives an upper bound (the *incumbent*).

Algorithm (sketch)

1. Solve the relaxed QP at a node.
2. If the solution is integer: update the incumbent.
3. If some δ_i is fractional (e.g. $\delta_i = 0.4$): branch into two sub-problems with $\delta_i = 0$ and $\delta_i = 1$ fixed.
4. Prune nodes that cannot beat the incumbent; repeat until no open nodes remain $\Rightarrow$ incumbent is optimal.

Branch and bound: a small example

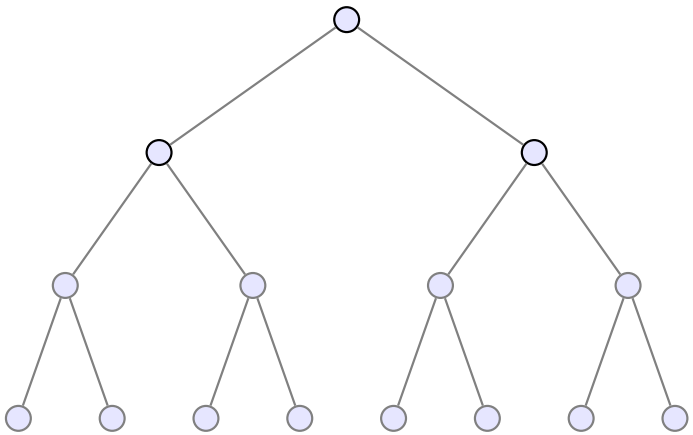


A node is fathomed (not explored further) if its relaxed QP is

- **infeasible** – no integer solution below it either;
- **integer** – best possible in that subtree, compare with incumbent;
- **bounded out** – lower bound $\geq$ incumbent, subtree cannot do better.

Lower bounds only increase going down the tree (more variables fixed).

Why is this hard? Combinatorial explosion



$n_b = 3$ binaries $\Rightarrow 2^3 = 8$ leaves

n_b	2^{n_b} leaves
10	$\approx 10^3$
30	$\approx 10^9$
60	$\approx 10^{18}$

- Full tree with n_b binaries has 2^{n_b} leaves (and $2^{n_b+1} - 1$ nodes, each a QP).
- Pruning often removes most of the tree . . . but in the worst case nothing can be pruned.
- MIQP / 0-1 integer programming is NP-hard

Each QP is polynomial – the number of QPs is exponential.

MI-MPC: the horizon makes the tree large

Integer decisions are repeated at *every* step of the horizon $k = 0, \ldots, N-1$. Worst-case number of integer combinations (= leaves of the B&B tree = QPs in the worst case):

Integer decisions per time step	# variables	worst case
1 binary, $\delta_k \in \{0,1\}$	N	$\mathcal{O}(2^N)$
m binaries, $\delta_k \in \{0,1\}^m$	mN	$\mathcal{O}(2^{mN})$
m integers, $\delta_k \in \{v_1, \ldots, v_p\}^m$	mN	$\mathcal{O}(p^{mN})$

- Total cost $\approx$ (# QPs) $\times$ (cost per QP) $= \mathcal{O}(p^{mN}) \cdot \mathrm{poly}(N)$: exponential in the horizon N.
- p-valued integers via binaries: $\lceil \log_2 p \rceil$ bits, or p one-hot binaries with $\sum_j \delta_{k,j} = 1$ – either way still p^{mN} feasible combinations.
- Example: $m = 3$, $N = 20 \Rightarrow 2^{60} \approx 10^{18}$ QPs; at 1 ms per QP $\approx$ 36 million years.

$\Rightarrow$ Real-time MI-MPC needs pruning, warm starts, move blocking, heuristics.

Approximations I: round-off

For real-time use we often *avoid* the full MIP and settle for a good-enough integer solution.

Rounding heuristic:

- Solve the continuous relaxation once (a single convex QP).
- Round each relaxed binary $\delta_i^\star$ to the nearest integer (0 or 1).
- Optionally re-solve for the continuous inputs u_c with the integers now fixed.

Pros

- Very fast: essentially one QP.
- Trivial to implement.

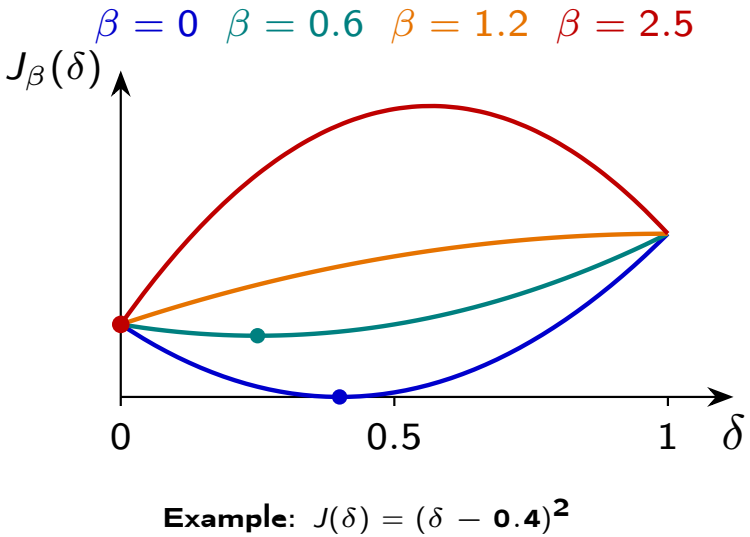
Cons

- Suboptimal; no optimality guarantee.
- Rounding can violate constraints.

Approximations II: penalty-term homotopy

Relax the binaries, $\delta_k \in \{0,1\}^m \;\rightarrow\; \delta_k \in [0,1]^m$, and penalize fractional values:

$$\mathcal{P}(\beta): \min_{u,\delta} \; J(u,\delta) + \beta \sum_{k=0}^{N-1} \sum_{j=1}^{m} \delta_{k,j}\,(1-\delta_{k,j}) \;\text{ s.t. MPC constraints, } \delta_k \in [0,1]^m$$



Example: $J(\delta) = (\delta - \mathbf{0.4})^{\mathbf{2}}$

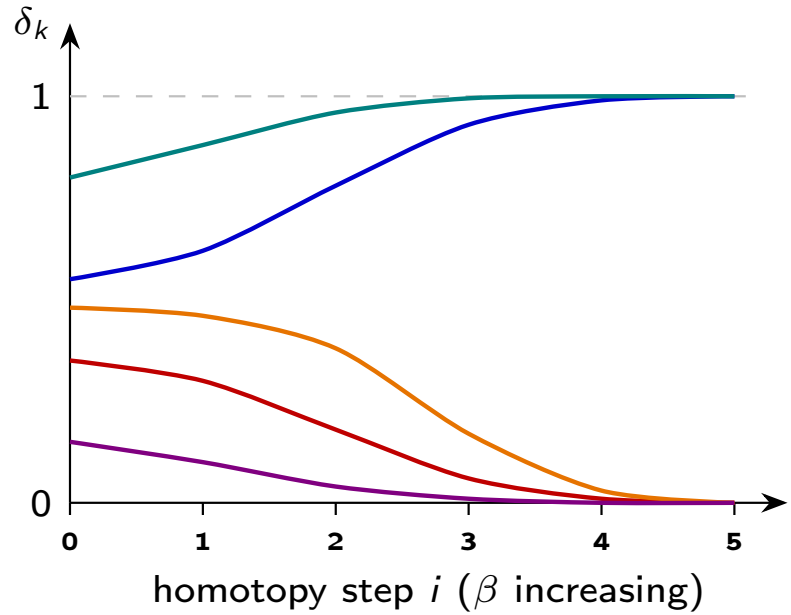
- $\delta(1-\delta) = 0$ *only* for $\delta \in \{0,1\}$; largest at $\delta = 0.5$.
- Penalty is concave: increasing β lifts the interior and pushes minimizers to integer points.
- $\beta = 0$: convex relaxation (lower bound).
- β large: minimizers are integer.

Catch: for $\beta > 0$ the problem is *nonconvex*, even if the MIQP was convex; a local NLP solver may end in a poor integer point.

Homotopy: increase β gradually

Jumping straight to a large β creates many local minima; the solver just rounds towards its initial guess. Instead, follow the solution path from the convex relaxation:

1. $\beta_0 = 0$: solve the relaxed QP $\rightarrow (u^0, \delta^0)$, J_{relax} is a lower bound.
2. For $i = 1, 2, \ldots$: $\beta_i = \rho\, \beta_{i-1}$ ($\rho \approx 2$–10); solve $\mathcal{P}(\beta_i)$ with a local NLP solver, warm-started at (u^{i-1}, δ^{i-1}).
3. Stop when $\max_{k,j} |\delta_{k,j} - \text{round}(\delta_{k,j})| \leq \varepsilon$.
4. Fix $\delta = \text{round}(\delta)$, solve the QP in u $\rightarrow$ feasible J_{int}.



Relaxed values are driven to $\{0, 1\}$

- Suboptimality certificate: $0 \leq J_{\text{int}} - J^{\star}_{\text{MIQP}} \leq J_{\text{int}} - J_{\text{relax}}$.
- Each step is a parametric NLP in β: sensitivity-based path following (tangential predictor) can replace full solves.

Penalty homotopy in MI-MPC: pros and cons

Advantages

- Cost $\approx$ #steps $\times$ one NLP: no exponential tree.
- Any standard NLP solver (IPOPT, SQP, ...).
- Warm start from the previous step *and* from the shifted solution of the last MPC sample.
- Also for nonlinear dynamics (MINLP); p-valued integers via one-hot binaries.

Limitations

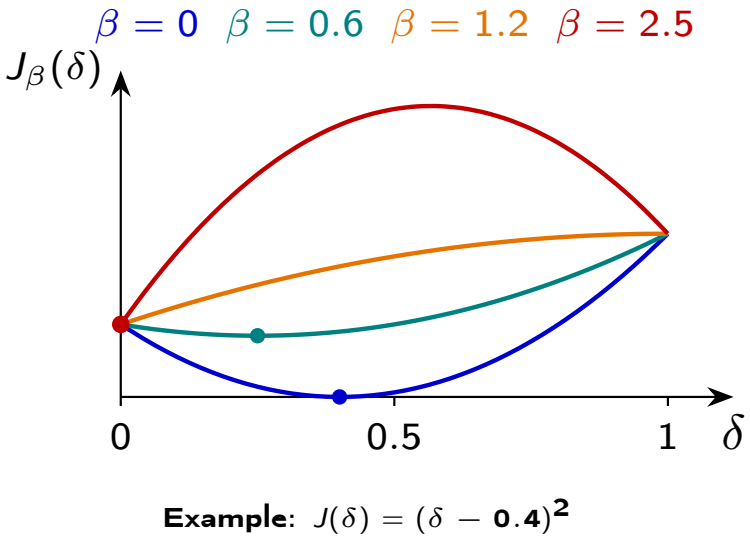
- A heuristic: no guarantee of the global integer optimum.
- Depends on the β-schedule and starting point.
- Large β: nonconvex, ill-conditioned NLPs.
- Rounded δ may be infeasible $\Rightarrow$ fallback (shifted previous solution).

	Branch and bound	**Penalty homotopy**
Worst-case cost	exponential, $\mathcal{O}(2^{mN})$ QPs	polynomial per NLP
Solution	global optimum (convex case)	local, integer feasible
Bound / gap	yes	$J_{\text{int}} - J_{\text{relax}}$

Penalty term homotopy: the idea

Relax the binaries, $\delta_k \in \{0,1\}^m \;\rightarrow\; \delta_k \in [0,1]^m$, and penalize fractional values:

$$\mathcal{P}(\beta):\ \min_{u,\delta}\ J(u,\delta)+\beta\sum_{k=0}^{N-1}\sum_{j=1}^{m}\delta_{k,j}\,(1-\delta_{k,j})\ \text{ s.t. MPC constraints, } \delta_k \in [0,1]^m$$



Example: $J(\delta) = (\delta - \mathbf{0.4})^{\mathbf{2}}$

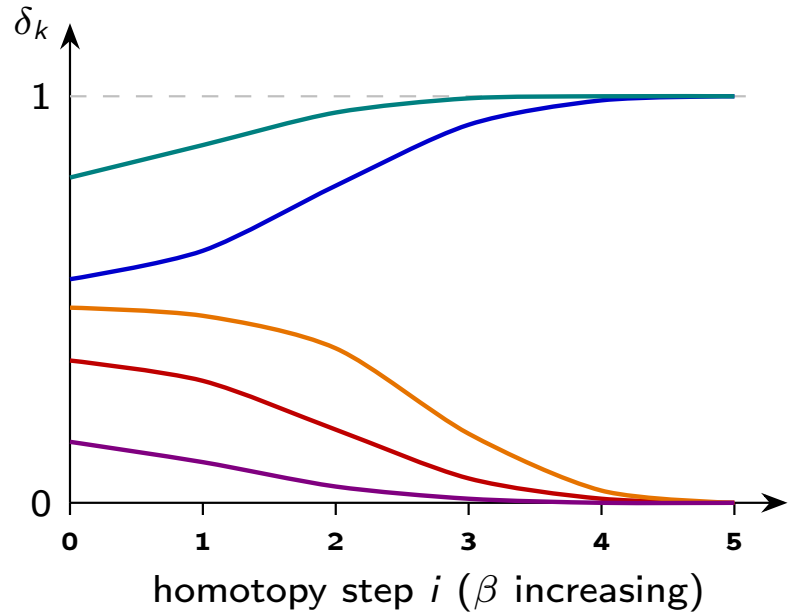
- $\delta(1-\delta) = 0$ *only* for $\delta \in \{0,1\}$; largest at $\delta = 0.5$.
- Penalty is concave: increasing β lifts the interior and pushes minimizers to integer points.
- $\beta = 0$: convex relaxation (lower bound).
- β large: minimizers are integer.

Catch: for $\beta > 0$ the problem is *nonconvex*, even if the MIQP was convex; a local NLP solver may end in a poor integer point.

Homotopy: increase β gradually

Jumping straight to a large β creates many local minima; the solver just rounds towards its initial guess. Instead, follow the solution path from the convex relaxation:

1. $\beta_0 = 0$: solve the relaxed QP $\rightarrow (u^0, \delta^0)$, J_{relax} is a lower bound.
2. For $i = 1, 2, \ldots$: $\beta_i = \rho\, \beta_{i-1}$ ($\rho \approx 2$–10); solve $\mathcal{P}(\beta_i)$ with a local NLP solver, warm-started at (u^{i-1}, δ^{i-1}).
3. Stop when $\max_{k,j} |\delta_{k,j} - \text{round}(\delta_{k,j})| \leq \varepsilon$.
4. Fix $\delta = \text{round}(\delta)$, solve the QP in u $\rightarrow$ feasible J_{int}.



Relaxed values are driven to $\{0, 1\}$

- Suboptimality certificate: $0 \leq J_{\text{int}} - J^{\star}_{\text{MIQP}} \leq J_{\text{int}} - J_{\text{relax}}$.
- Each step is a parametric NLP in β: sensitivity-based path following (tangential predictor) can replace full solves.

Penalty homotopy in MI-MPC: pros and cons

Advantages

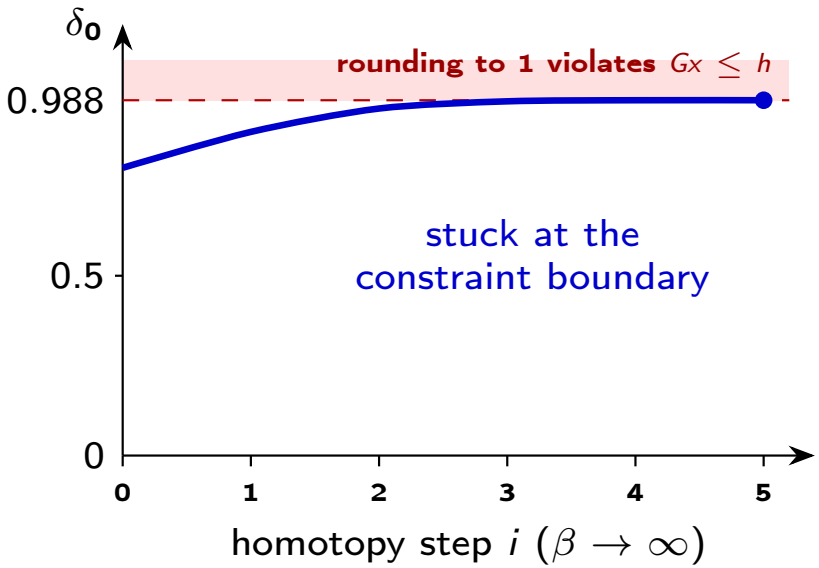
- Cost $\approx$ #steps $\times$ one NLP: no exponential tree.
- Any standard NLP solver (IPOPT, SQP, ...).
- Warm start from the previous step *and* from the shifted solution of the last MPC sample.
- Also for nonlinear dynamics (MINLP); p-valued integers via one-hot binaries.

Limitations

- A heuristic: no guarantee of the global integer optimum.
- Depends on the β-schedule and starting point.
- Large β: nonconvex, ill-conditioned NLPs.
- With state constraints, δ can get stuck at fractional values (next slides).

	Branch and bound	**Penalty homotopy**
Worst-case cost	exponential, $\mathcal{O}(2^{mN})$ QPs	polynomial per NLP
Solution	global optimum (convex case)	local, integer feasible
Bound / gap	yes	$J_{\text{int}} - J_{\text{relax}}$

Pitfall: penalty homotopy with state constraints



- The relaxed solution wants $\delta_0 \to 1$ (e.g. fire a pellet), but a state constraint $Gx_k \leq h$ only allows $\delta_0 \leq 0.988$.
- At $\delta_0 = 0.988$ the penalty gradient $\beta(1 - 2\delta_0) \approx -0.98\,\beta$ pushes towards 1, into the active constraint.
- Increasing $\beta \to \infty$ only pushes harder: $\delta_0 = 0.988$ remains a local minimizer.
- $\delta_0 = 1$ is infeasible, so it is never reached. The *feasible* integer $\delta_0 = 0$ lies on the other side of the penalty's hump, and a warm-started local solver never gets there.

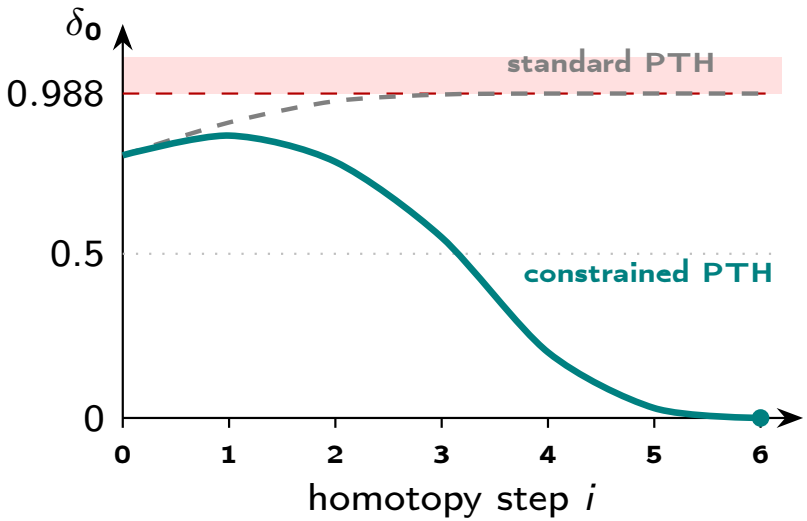
We need to push the solution away from the constraint, towards $\delta_0 = 0$.

Constrained penalty term homotopy

Add a logarithmic barrier on the state constraints, grown with the penalty (s.t. MPC constraints, $\delta_k \in [0, 1]$):

$$\min_{u,\delta} \; J(u,\delta) + \beta_j \sum_{k=0}^{N-1} \delta_k(1-\delta_k) \; - \; \gamma_j \sum_{k=0}^{N} \ln\big(h - Gx_k\big)$$

- Barrier $\to \infty$ at the boundary: states are pushed into the interior.
- $\delta_0 = 0.988$ (state on the boundary) becomes expensive, so δ_0 decreases; below 0.5 the penalty drives it to the feasible integer 0.
- Grow both: $\beta_j = \beta_{\text{init}} \beta_{\text{inc}}^{j-1}$, $\gamma_j = \beta_{j+1}$. ($\gamma \equiv 0$: standard PTH.)
- Stop when all δ_k are ε-close to $\{0, 1\}$ (rounding keeps $Gx \le h$); fallback $\delta = 0$ after $j_{\max}$ steps.



C. A. Orrico, M. van Berkel, T. Bosman, W. P. M. H. Heemels, D. Krishnamoorthy, "Mixed-integer MPC strategies for fueling and density control in fusion tokamaks," *IEEE Control Systems Letters*, vol. 7, pp. 1897–1902, 2023. (IEEE CSS TC-ES Outstanding Student Paper Award)

More solution strategies

More solution stratgies using Machine learning techniques:

- Learn the policy
- Learn the cost-to-go
- Learn the branching strategies

  We will see some of these in the *Learning-based MPC* lecture

MPC can be complex also without integer decisions

- Forget integer decision variables
- Solving MPC problems with only real decision variables can be complex for many applications
- must solve an optimization problem at each sampling instant
- for NMPC, this involves solving nonlinear (nonconvex) optimization problems.

Can we reduce the complexity of solving MPC problems online?

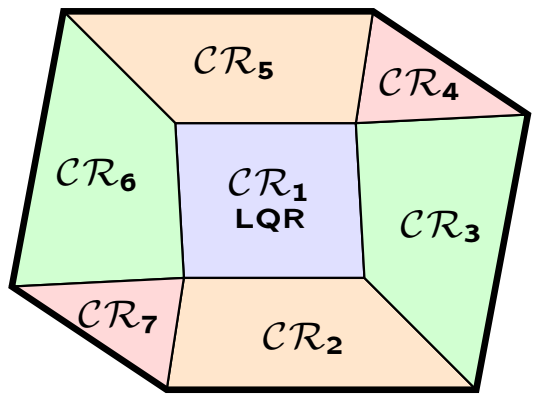
Explicit MPC: solve the QP for *all* states offline

Linear MPC, after eliminating the states, is a multiparametric QP (mpQP) in the state x:

$$U^\star(x) = \arg\min_U \ \tfrac{1}{2} U^\top H U + x^\top F U \quad \text{s.t.} \quad GU \le W + Sx$$

Theorem (Bemporad et al., 2002)

$u^\star(x)$ is continuous, piecewise affine on a partition of the feasible set into polyhedral *critical regions*:



$$u^\star(x) = K_i x + g_i \quad \text{for} \quad x \in \mathcal{CR}_i = \{x : E_i x \le e_i\}$$

- Each critical region corresponds to one optimal active set; $\mathcal{CR}_1$ has no active constraints, i.e. $u = K_{\text{LQR}}\, x$.
- With the active set fixed, the KKT conditions are *linear* in $x \Rightarrow$ the solution is affine in x: the sensitivity $\partial u^\star / \partial x$ is exact and constant within a region.

Explicit MPC: offline vs. online

Offline (once)

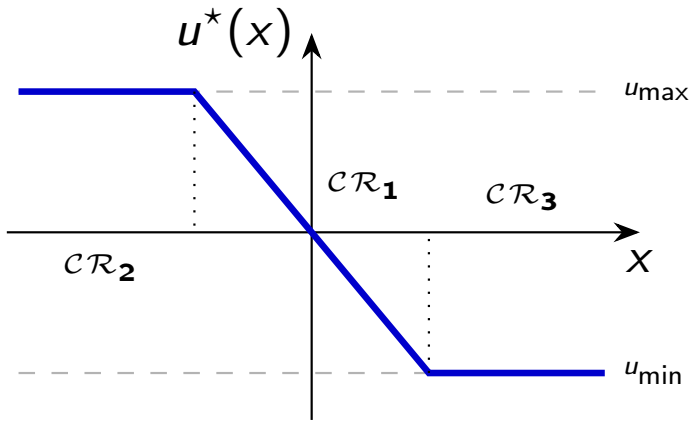
Solve the mpQP: find all regions $\mathcal{CR}_i$ and affine laws, and store the table $\{E_i, e_i, K_i, g_i\}_{i=1}^{n_r}$.

Advantages

- No online optimization: a lookup and a matrix–vector product, in microseconds, on cheap embedded hardware.
- Known worst-case computation time; the control law can be inspected and verified offline.
- Point location: sequential search $\mathcal{O}(n_r)$, or a binary search tree $\mathcal{O}(\log n_r)$ (Tøndel et al., 2003).

Online (every sample)

Point location: find i with $x \in \mathcal{CR}_i$, then apply $u = K_i x + g_i$.



Scalar example: explicit MPC = saturated linear feedback

Explicit MPC: the catch

1. **The number of regions explodes**
   - $n_r \leq$ number of possible optimal active sets $\leq \sum_{k=0}^{Nn_u} \binom{q}{k}$, with $q \propto N$ constraints over the horizon $\Rightarrow$ exponential in N and in the number of constraints.
   - Grows fast with n_x and n_u (offline time, memory, point location). Rule of thumb: only small problems, $n_x \lesssim 5$, few inputs, short horizons. References or disturbances add parameters.
2. **Only for linear (or piecewise-affine) models**
   - The PWA structure comes from linear KKT conditions. For a nonlinear model, $u^\star(x)$ is not PWA and the regions are not polyhedral $\Rightarrow$ only *approximate* explicit NMPC.
   - Hybrid systems (mp-MIQP): non-convex partitions, even more regions.
3. **No flexibility online**
   - Any change in model, weights, constraints or horizon: recompute offline.

Remedies: region merging, approximate/simplicial explicit MPC, partial enumeration, neural-network approximations of the PWA law.

Motivation: NMPC is a parametric NLP

At every sampling time t_k: measure $\hat{x}_k$, solve, apply the first input.

$$\mathcal{P}(\hat{x}_k): \quad \min_{x,u} \sum_{i=0}^{N-1} \ell(x_i, u_i) + V_f(x_N) \quad \text{s.t.} \quad x_0 = \hat{x}_k, \quad x_{i+1} = f(x_i, u_i), \quad h(x_i, u_i) \leq 0$$

- Solving the NLP takes time $T_{\text{comp}} \Rightarrow$ the input is based on an outdated state (computational delay), and T_{comp} may even exceed the sampling time.
- But $\mathcal{P}(\hat{x}_k)$ and $\mathcal{P}(\hat{x}_{k+1})$ differ *only* in the parameter $p = \hat{x}$; their solutions are close.

Questions

1. How does the optimal solution $s^\star(p)$ change with p? $\rightarrow$ NLP sensitivity
2. Can we do the expensive work *before* the measurement arrives? $\rightarrow$ RTI and advanced-step NMPC

Parametric NLP and its KKT conditions

$$\mathcal{P}(p): \quad \min_{w} \; J(w,p) \quad \text{s.t.} \quad c(w,p)=0, \;\; h(w,p)\le 0$$

Lagrangian $\mathcal{L}(w,\lambda,\mu,p) = J + \lambda^\top c + \mu^\top h$, primal-dual vector $s=(w,\lambda,\mu)$.

KKT conditions: $\nabla_w \mathcal{L}=0, \;\; c=0, \;\; h\le 0, \;\; \mu\ge 0, \;\; \mu_i h_i = 0$

For a *fixed active set* $\mathcal{A}=\{i : h_i(w^\star,p)=0\}$ the KKT conditions are a smooth system of equations

$$F(s,p)=\begin{bmatrix}\nabla_w \mathcal{L}(w,\lambda,\mu,p)\\ c(w,p)\\ h_{\mathcal{A}}(w,p)\end{bmatrix}=0, \qquad \mu_{\mathcal{A}}\ge 0, \;\; \mu_{i\notin\mathcal{A}}=0$$

with Jacobian (the KKT matrix)

$$K=\frac{\partial F}{\partial s}=\begin{bmatrix}\nabla^2_{ww}\mathcal{L} & \nabla_w c & \nabla_w h_{\mathcal{A}}\\ \nabla_w c^\top & 0 & 0\\ \nabla_w h_{\mathcal{A}}^\top & 0 & 0\end{bmatrix}$$

$\Rightarrow$ apply the implicit function theorem to $F(s^\star(p),p)=0$.

NLP sensitivity theorem

Theorem (Fiacco, 1976)

Let $s^\star(p_0)$ be a KKT point of $\mathcal{P}(p_0)$ satisfying **LICQ**, **SOSC** and **strict complementarity**, with J, c, h twice continuously differentiable. Then

- K is nonsingular and $w^\star(p_0)$ is a strict local minimizer;
- for p near p_0 there is a unique, continuously differentiable $s^\star(p)$ with the *same active set*;
- its sensitivity is

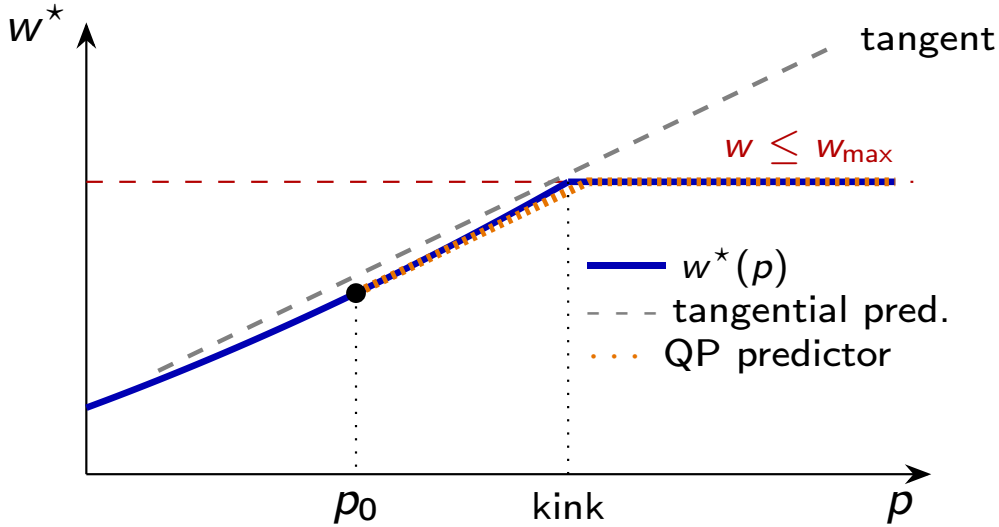
$$\frac{\partial s^\star}{\partial p} = -K^{-1}\,\frac{\partial F}{\partial p}.$$

Tangential predictor: for a new parameter $p = p_0 + \Delta p$

$$s^\star(p) \;\approx\; s^\star(p_0) \;-\; K^{-1}\frac{\partial F}{\partial p}\,\Delta p, \qquad \text{error } \mathcal{O}\big(\|\Delta p\|^2\big)$$

- One linear solve with the KKT matrix: no new NLP needed.
- K is often already factorized by the NLP solver (e.g. IPOPT) $\Rightarrow$ the update is just a back substitution.

When the active set changes



- At an active-set change, strict complementarity fails: $s^\star(p)$ has a kink.
- The linear predictor keeps the old active set and violates constraints.
- Fix: keep the inequalities $\Rightarrow$ solve a QP instead of a linear system.

QP predictor (linearize at $s_0 = s^\star(p_0)$, new $p = p_0 + \Delta p$):

$$\min_{\Delta w} \tfrac{1}{2}\Delta w^\top \nabla^2_{ww}\mathcal{L}\,\Delta w + \nabla_w J^\top \Delta w + \Delta p^\top \nabla^2_{pw}\mathcal{L}\,\Delta w$$

$$\text{s.t. } c + \nabla_w c^\top \Delta w + \nabla_p c^\top \Delta p = 0, \quad h + \nabla_w h^\top \Delta w + \nabla_p h^\top \Delta p \le 0$$

Still first-order accurate across active-set changes (under strong regularity, Robinson 1980): $s^\star(p)$ is piecewise smooth.

Key MPC structure: initial-value embedding

Keep x_0 as a decision variable; the parameter enters linearly in one constraint:

$$x_0 - \hat{x} = 0, \qquad x_{i+1} - f(x_i, u_i) = 0, \qquad h(x_i, u_i) \leq 0$$

- $\dfrac{\partial F}{\partial p} = \begin{bmatrix} 0 & -I & 0 & \cdots \end{bmatrix}^\top$ is constant, and $\nabla^2_{pw}\mathcal{L} = 0$.
- Hence $F(s_0, p_1) = F(s_0, p_0) + \dfrac{\partial F}{\partial p}\Delta p$ exactly.

Consequence: one Newton step = predictor (+ corrector)

A Newton step on $\mathcal{P}(p_1)$, started at the old solution $s_0 = s^\star(p_0)$:

$$s_1 = s_0 - K^{-1}F(s_0, p_1) = s_0 - K^{-1}\frac{\partial F}{\partial p}\Delta p \quad = \text{tangential predictor.}$$

If s_0 is *not* optimal, the same step also contains a Newton corrector towards $s^\star$.
With inequalities: one SQP step = QP predictor.

- All linearizations, Hessians and K depend on s_0 only, not on $\hat{x} \Rightarrow$ they can be computed before $\hat{x}$ is measured.

Real-time iteration (RTI)

Idea: do only one SQP iteration per sample, split into two phases, and let the iterates converge *over time* while the system moves.

Preparation phase (before t_k)

- Shift previous iterate $\rightarrow$ linearization point $\bar{w}$
- Simulate f, compute Jacobians, Gauss–Newton Hessian
- Build (and condense) the QP with $\hat{x}$ left as a parameter

Feedback phase (at t_k)

- Measure $\hat{x}_k$, insert into $x_0 + \Delta x_0 = \hat{x}_k$
- Solve one QP $\rightarrow \Delta w$
- Apply $u_0 = \bar{u}_0 + \Delta u_0$ immediately
- Update $\bar{w} \leftarrow \bar{w} + \Delta w$

- Feedback delay = one QP solve, comparable to linear MPC.
- The QP is a generalized tangential predictor from $\bar{w}$ to the new $\hat{x}$, plus a Newton corrector; it handles active-set changes.
- Price: never fully converged. Works well if the sampling is fast relative to the nonlinearity; stability results exist for this case (Diehl et al., 2005).

Advanced-step NMPC (asNMPC)

Idea: use the sampling interval to solve the full NLP for the predicted next state; correct with sensitivities when the measurement arrives.

Background (during $[t_k, t_{k+1}]$)

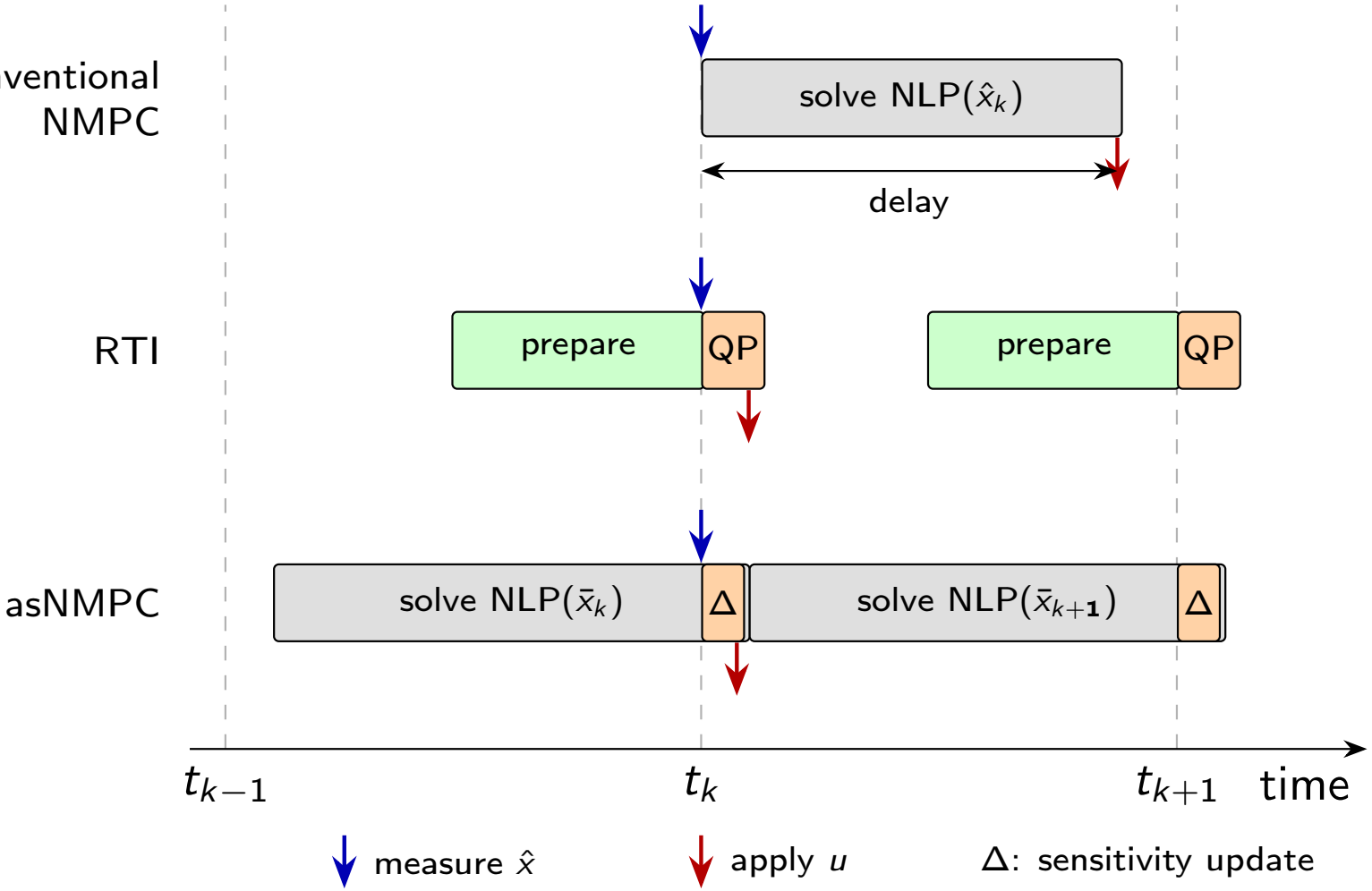
1. Predict $\bar{x}_{k+1} = f(\hat{x}_k, u_k)$
2. Solve $\mathcal{P}(\bar{x}_{k+1})$ to convergence $\rightarrow s^\star(\bar{x}_{k+1})$
3. Keep the factorized KKT matrix K

Online (at t_{k+1})

4. Measure $\hat{x}_{k+1}$; $\Delta p = \hat{x}_{k+1} - \bar{x}_{k+1}$
5. $s \approx s^\star(\bar{x}_{k+1}) - K^{-1} \frac{\partial F}{\partial p} \Delta p$ (one back substitution, typically ms)
6. Apply u_0 from s

- Error $\mathcal{O}(\|\hat{x}_{k+1} - \bar{x}_{k+1}\|^2)$: small when the model prediction is good; nominal stability and robustness carry over from ideal NMPC (Zavala & Biegler, 2009).
- Active-set changes: clipping, or a QP / path-following predictor (e.g. Jäschke, Yang & Biegler, 2014).
- Requires the full NLP to be solved within one sampling interval.

Timing comparison



- Both RTI and asNMPC move the expensive work before the measurement and use a sensitivity update once it arrives.

RTI vs. advanced-step NMPC

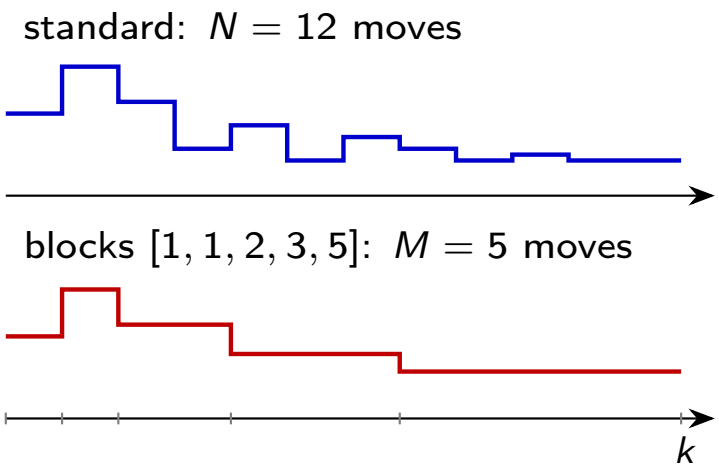
	RTI	**asNMPC**
Background work	linearize, build QP (one SQP iteration)	full NLP at predicted $\bar{x}$
Online work	one QP	one back substitution (+ active-set fix)
Linearized at	previous iterate (not converged)	converged $s^\star(\bar{x})$
Online result	predictor + one corrector step	predictor, error $\mathcal{O}(\|\Delta p\|^2)$
Limits sampling time	QP + preparation	full NLP solve
Active-set changes	handled by the QP	needs extra treatment

Common idea: NMPC solves a family of NLPs $\mathcal{P}(p)$, $p = \hat{x}$. The sensitivity $\partial s^\star / \partial p$ links neighbouring problems, so we can prepare ahead and correct in real time.

Move blocking: fewer input decisions

Hold the input constant over blocks of the horizon:

$$U = [u_0; \ldots; u_{N-1}] = (T \otimes I_{n_u})\,\tilde{U}, \qquad T \in \{0,1\}^{N \times M}, \quad M \ll N$$



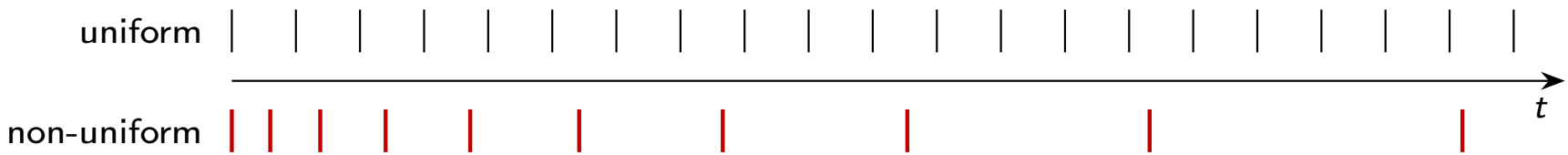
- Decision variables: $Nn_u \rightarrow Mn_u$.
- Model and state constraints are still evaluated at *every* k.
- Short blocks early (only u_0 is applied), long blocks later.
- Variants: input, offset and Δu blocking.

Catch: the shifted solution no longer has the blocked structure $\Rightarrow$ recursive feasibility and stability guarantees are lost.

Non-uniform time grid

Fine steps where accuracy matters, coarse steps further out: a long prediction time with few shooting intervals.

$$x_{i+1} = F(x_i, u_i, \Delta t_i), \qquad \Delta t_i = \Delta t_0 \, \rho^{\,i} \quad (\rho > 1)$$



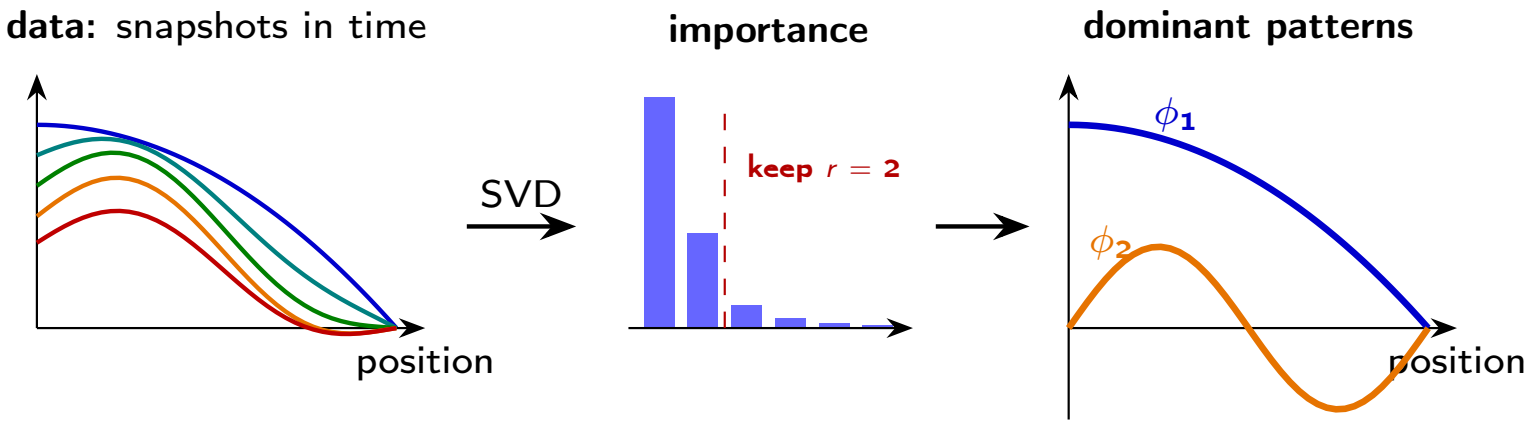
- Example: $T = 10\,\mathrm{s}$ at $\Delta t = 0.1\,\mathrm{s}$ needs 100 intervals; with $\Delta t_0 = 0.1\,\mathrm{s}$, $\rho = 1.3$ only 14.
- Reduces *states and inputs*; move blocking reduces only the inputs.
- Weight stage costs by Δt_i so long intervals are not under-weighted.

Catches:

- Constraints only checked at grid points: violations in between.
- Grid does not align after a shift: warm starts need interpolation; standard stability proofs do not apply.
- Long intervals need an accurate (adaptive) integrator.

Model reduction: dynamic mode decomposition (DMD)

Intuition: a large system (e.g. a spatial profile with 1000 grid points) usually moves in only a few dominant patterns.



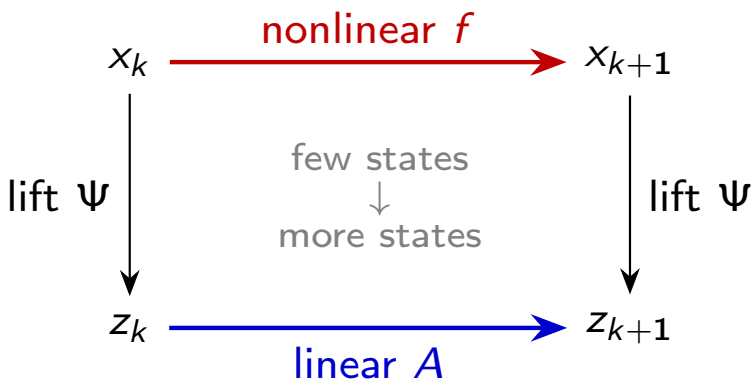
$$\underbrace{x(t)}_{\text{1000 numbers}} \approx z_1(t)\,\phi_1 + z_2(t)\,\phi_2, \qquad \underbrace{z(t)}_{\text{2 numbers}} \text{ evolves (approx.) linearly}$$

Recipe: snapshots of $x, u \to$ dominant patterns (SVD) $\to$ fit a linear model for the weights, $z_{k+1} \approx \tilde{A} z_k + \tilde{B} u_k$ (DMD, least squares) $\to$ linear MPC (a small QP) on z.

Catches: only valid near the data; needs well-excited data; model error must be handled (robust / offset-free MPC).

Koopman lifting: make nonlinear dynamics look linear

Intuition: add cleverly chosen extra states (*observables*) so that, in the new coordinates, the dynamics become linear.



Like feature engineering: trade *few nonlinear* states for *more linear* ones.

Example (exact):

$$x_1^+ = 0.9\, x_1$$
$$x_2^+ = 0.5\, x_2 + x_1^2$$

Add $z_3 = x_1^2$: since
$z_3^+ = (0.9 x_1)^2 = 0.81\, z_3$,

$$\begin{bmatrix} x_1 \\ x_2 \\ z_3 \end{bmatrix}^+ = \begin{bmatrix} 0.9 & 0 & 0 \\ 0 & 0.5 & 1 \\ 0 & 0 & 0.81 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ z_3 \end{bmatrix}$$

2 nonlinear $\rightarrow$ 3 linear states

In practice: many candidate observables (x, x^2, $\sin x$, RBFs, ...), linear fit on lifted data (DMD again), then *lift* $\hat{x} \rightarrow$ *predict linearly* $\rightarrow$ *QP*.
Catches: an exact finite lifting is rare $\Rightarrow$ prediction error grows along the horizon; inputs are assumed to act linearly in the lifted space.

Complexity reduction in Model Predictive Control

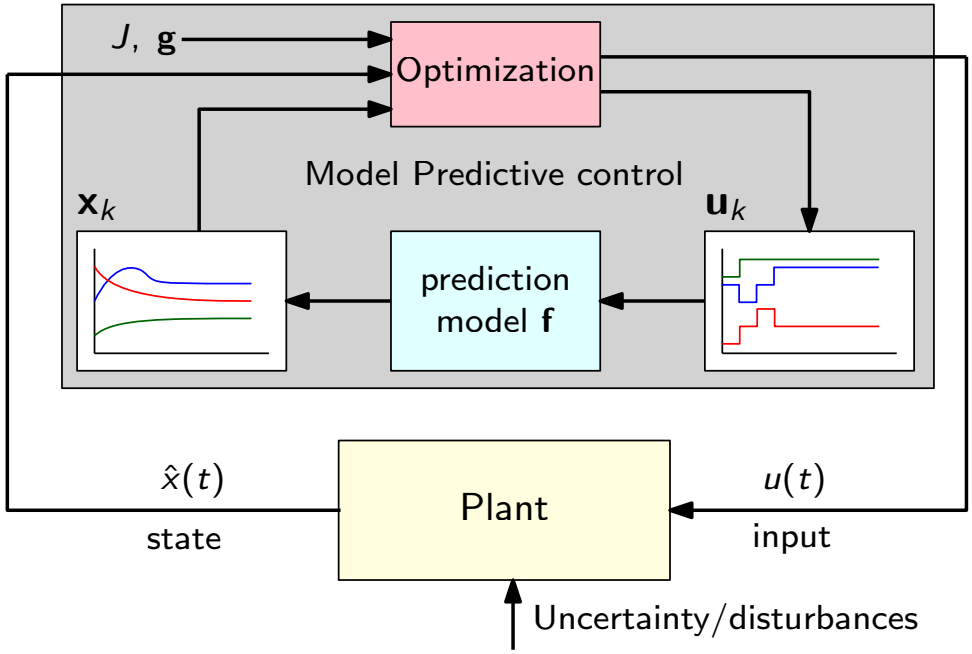
Using learning-based methods

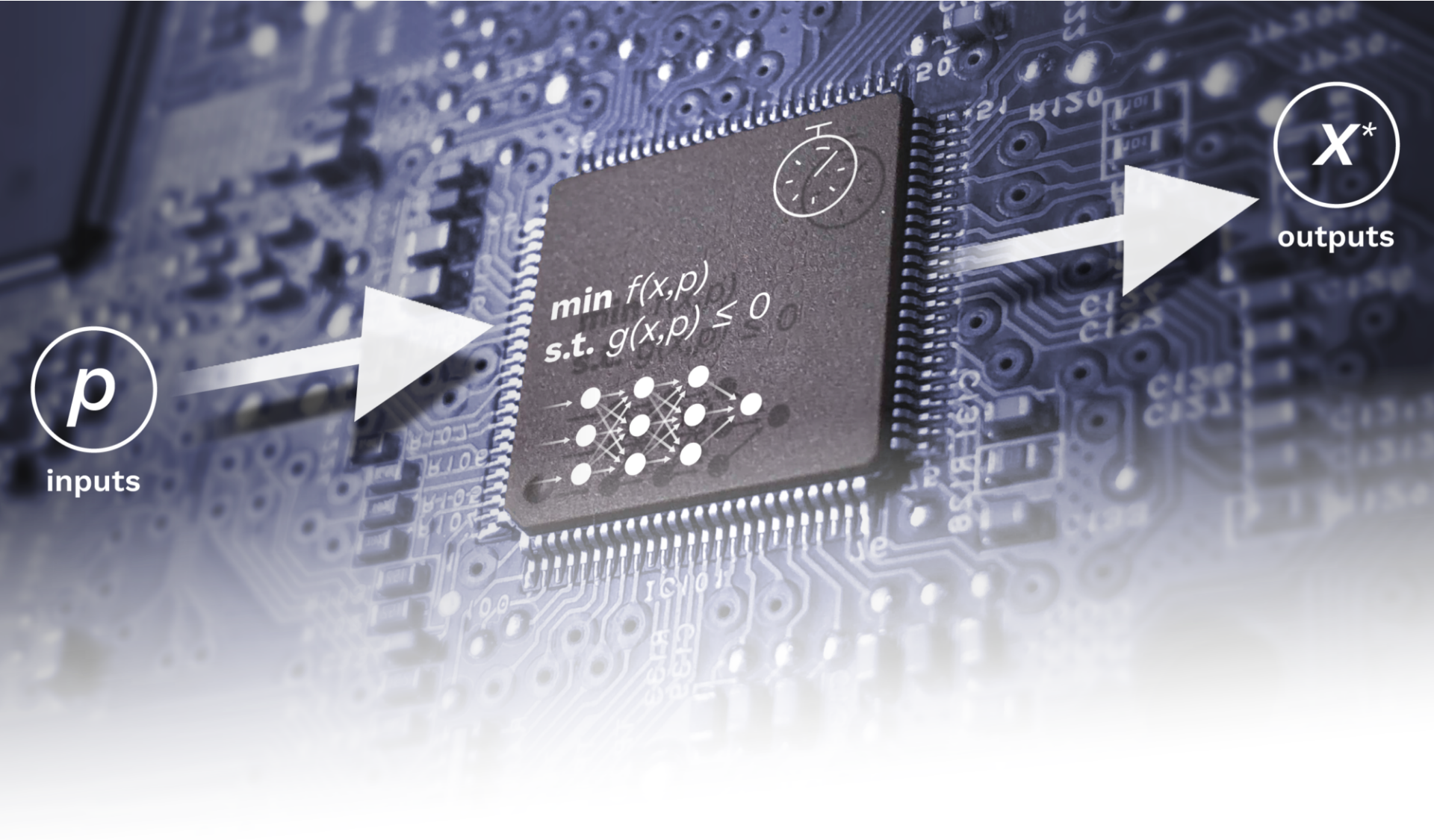
Dinesh Krishnamoorthy

NTNU

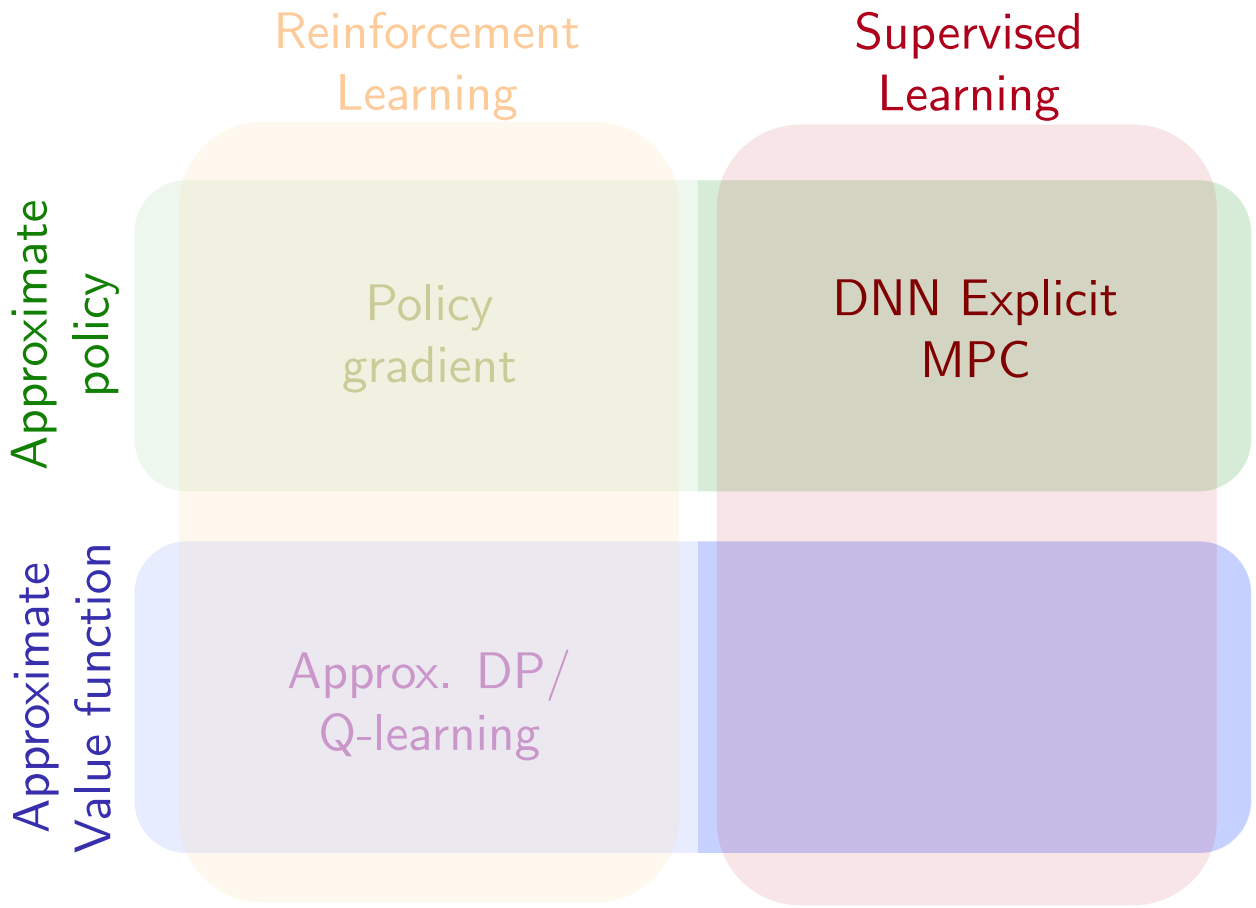
Outline

- Motivation - online complexity
- Learning-based components
- Reflections

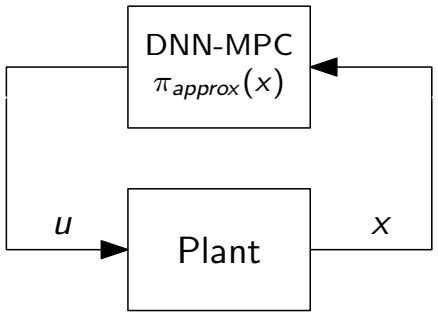


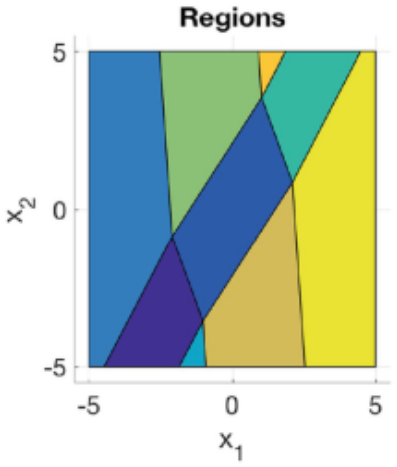


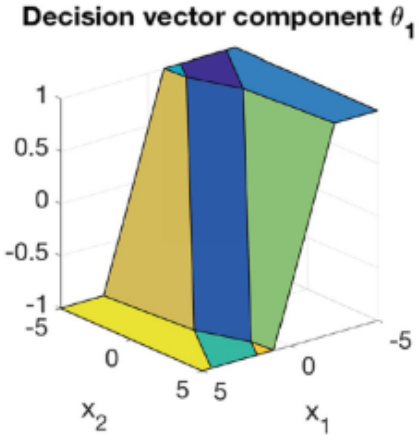
Premise: We have a good MPC problem formulation, but it is too complex for real-time implementation

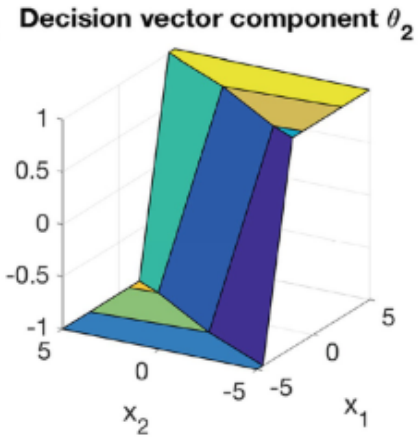

Reinforcement Learning
Supervised Learning
Approximate policy
Approximate Value function
Policy gradient
DNN Explicit MPC
Approx. DP/ Q-learning

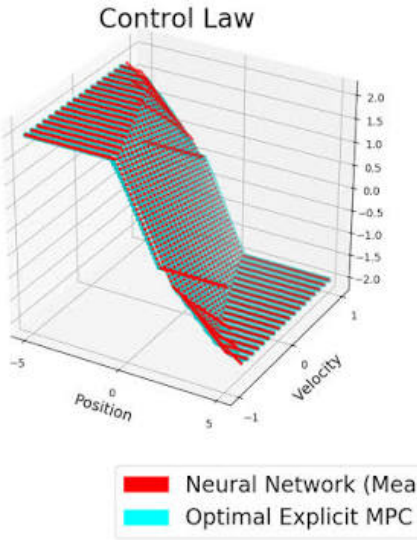
Learn optimal policy from expert data

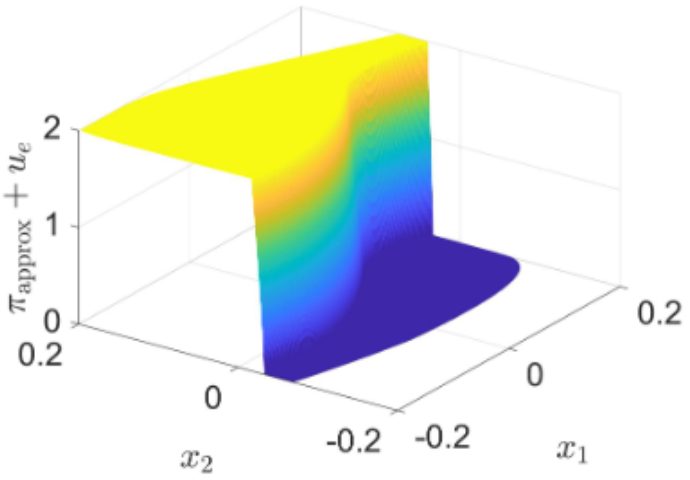




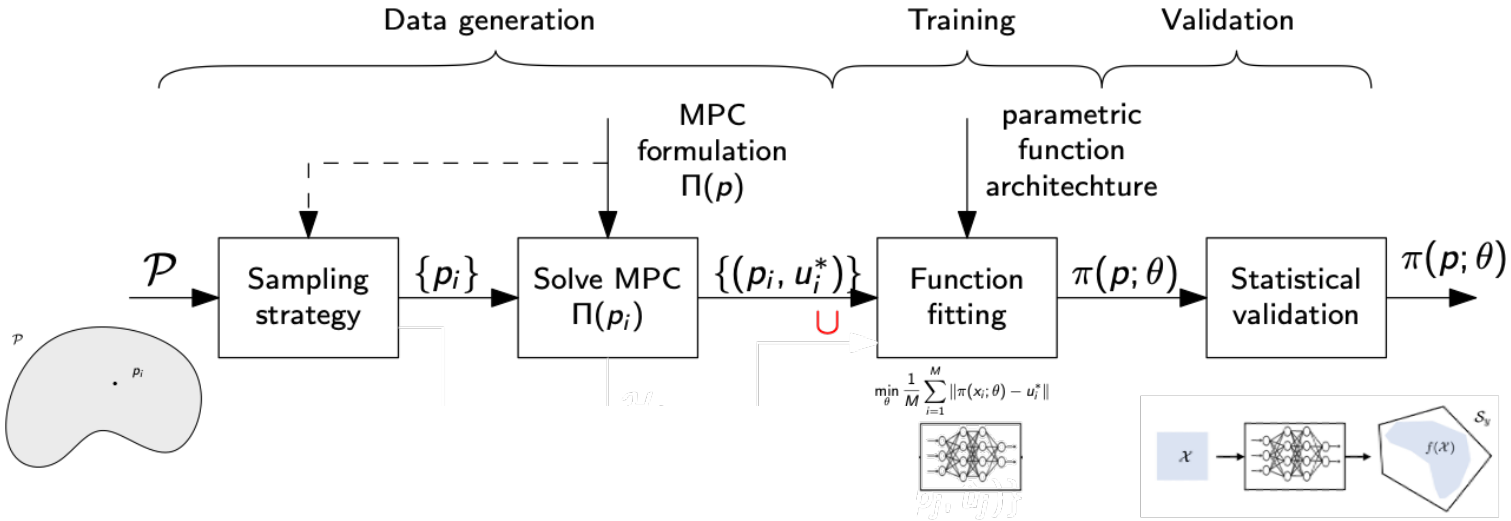


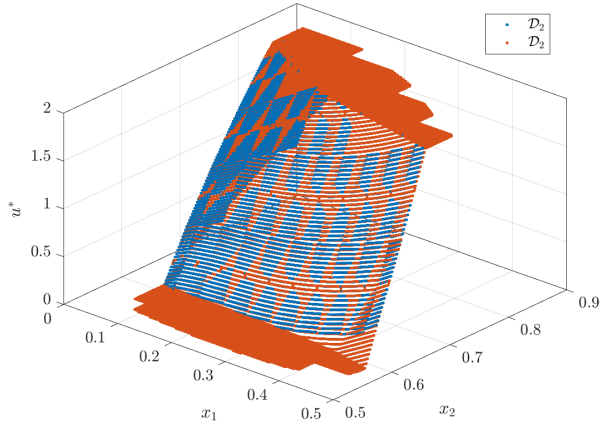


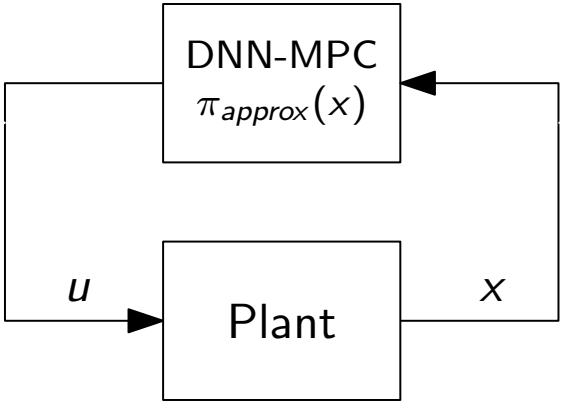




Learn optimal policy from expert data

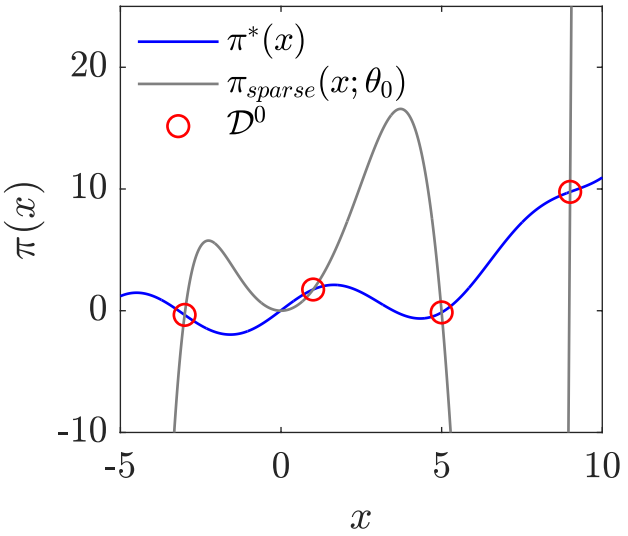


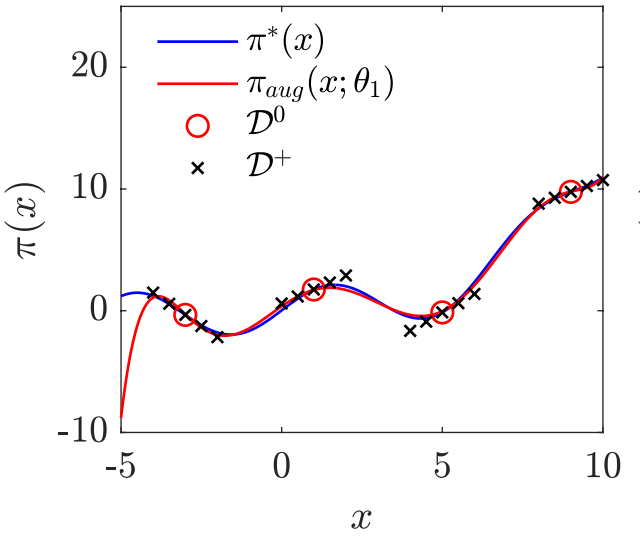


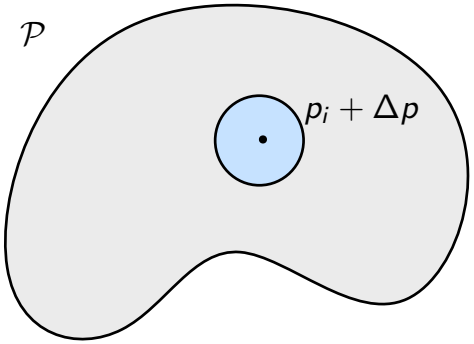


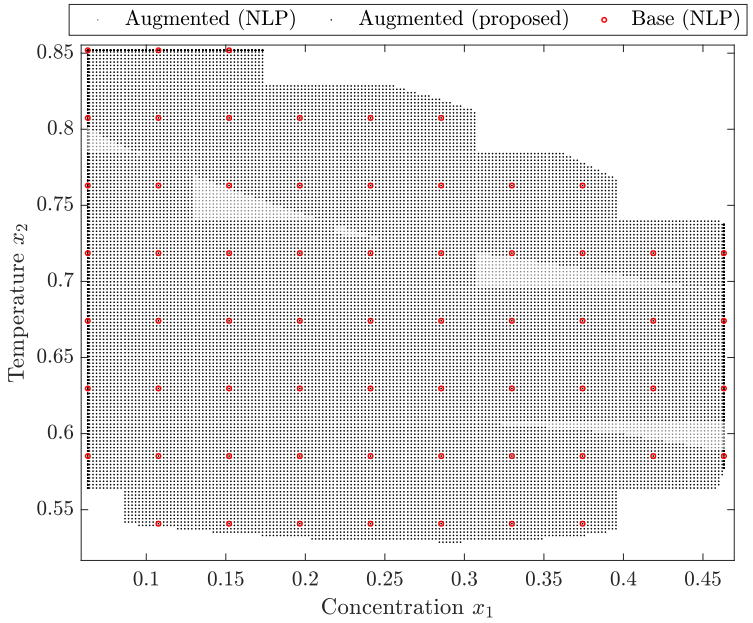
Parisini & Zoppoli (1995) *Automatica*, Chakrabarty et al. (2017) *IEEE TAC*, Karg & Lucia (2018), *IFAC NMPC*, Chen et al. (2018) *ACC* Chen et al. (2022) *Automatica*, Paulson & Mesbah (2020) *IEEE L-CSS*, Hertneck et al. (2018) *IEEE L-CSS*, Drgona et al. (2018) *Applied Energy*, Ceskö et al. (2015) *IEEE-TCST*, Zhang et al. (2019) *ACC*, Spielberg et al. (2019) *AIChE J.*

Sensitivity-based Data Augmentation









Krishnamoorthy, *IEEE Trans. Aut. Control.* (2021)

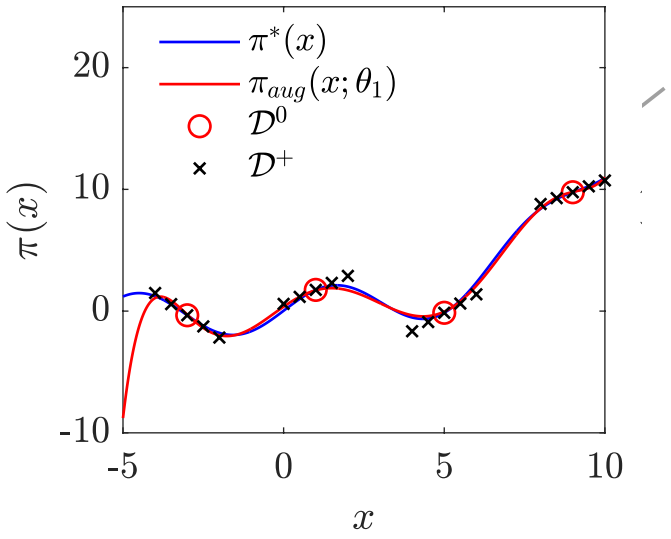
Sensitivity-based Data Augmentation

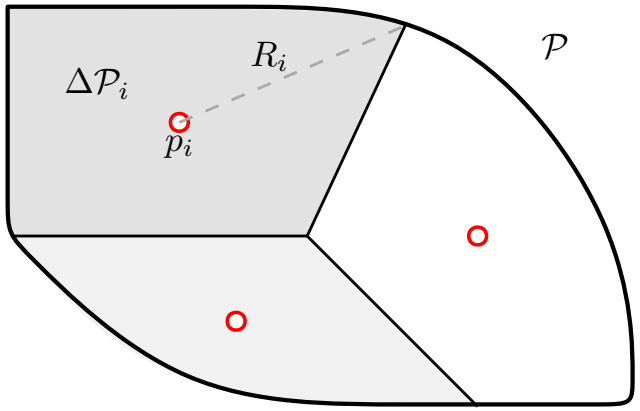
Error Bound due to augmenting with inexact samples

Assuming that the parametric function $\pi(x, \theta)$ is sufficiently richly parameterized, we have

$$\|\pi(x, \theta_1) - \pi^*(x)\| \leq \max\left(\left\{L_{x_i} R_i^2\right\}_{i=1}^{N}\right)$$

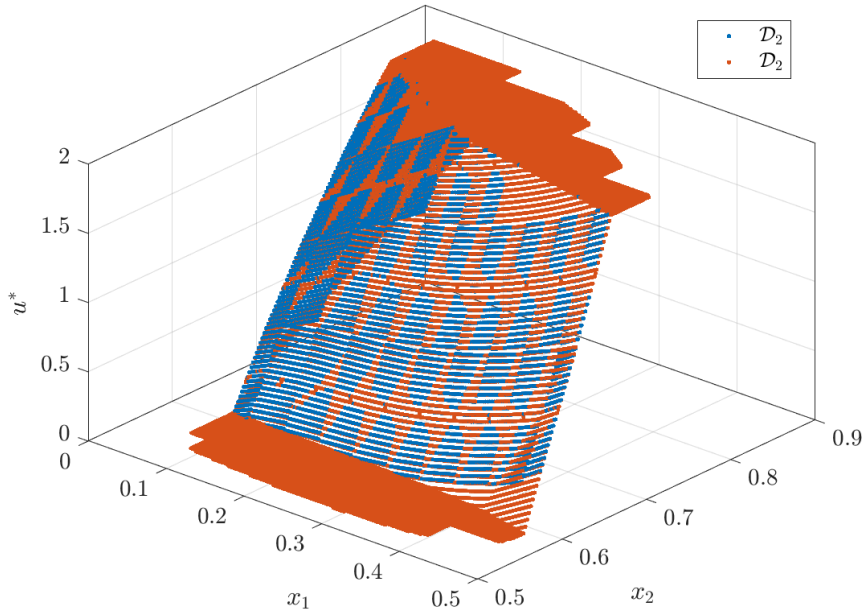
in probability as $M \to \infty$ if θ_1 is a consistent estimator.

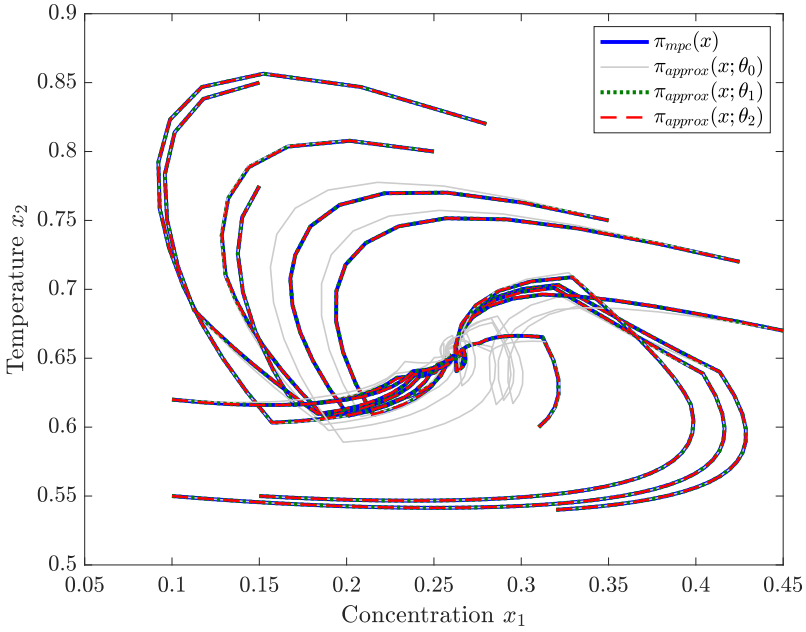




Krishnamoorthy, *IEEE Trans. Aut. Control.* (2021)

Example: Benchmark CSTR





Krishnamoorthy, *IEEE Trans. Aut. Control.* (2021)

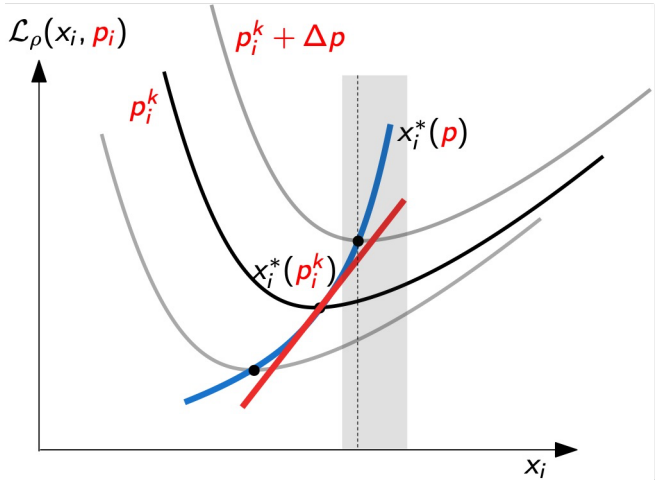
Sensitivity-based Data Augmentation

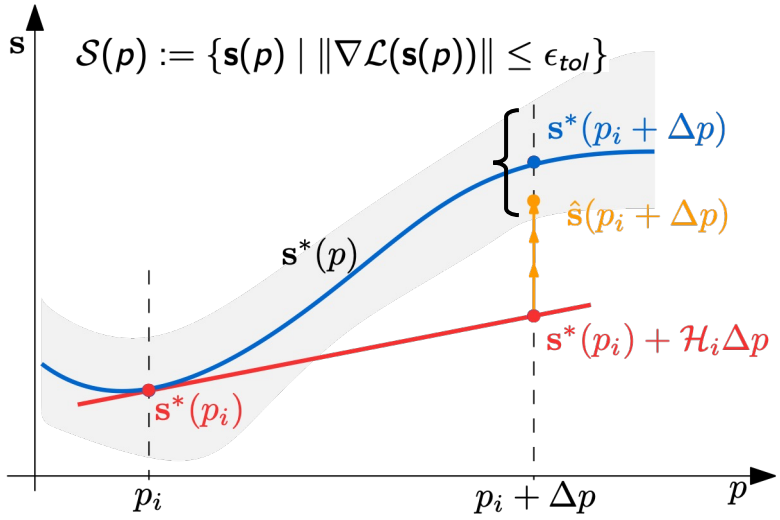
Error Bound due to augmenting with inexact samples

Assuming that the parametric function $\pi(x, \theta)$ is sufficiently richly parameterized, we have

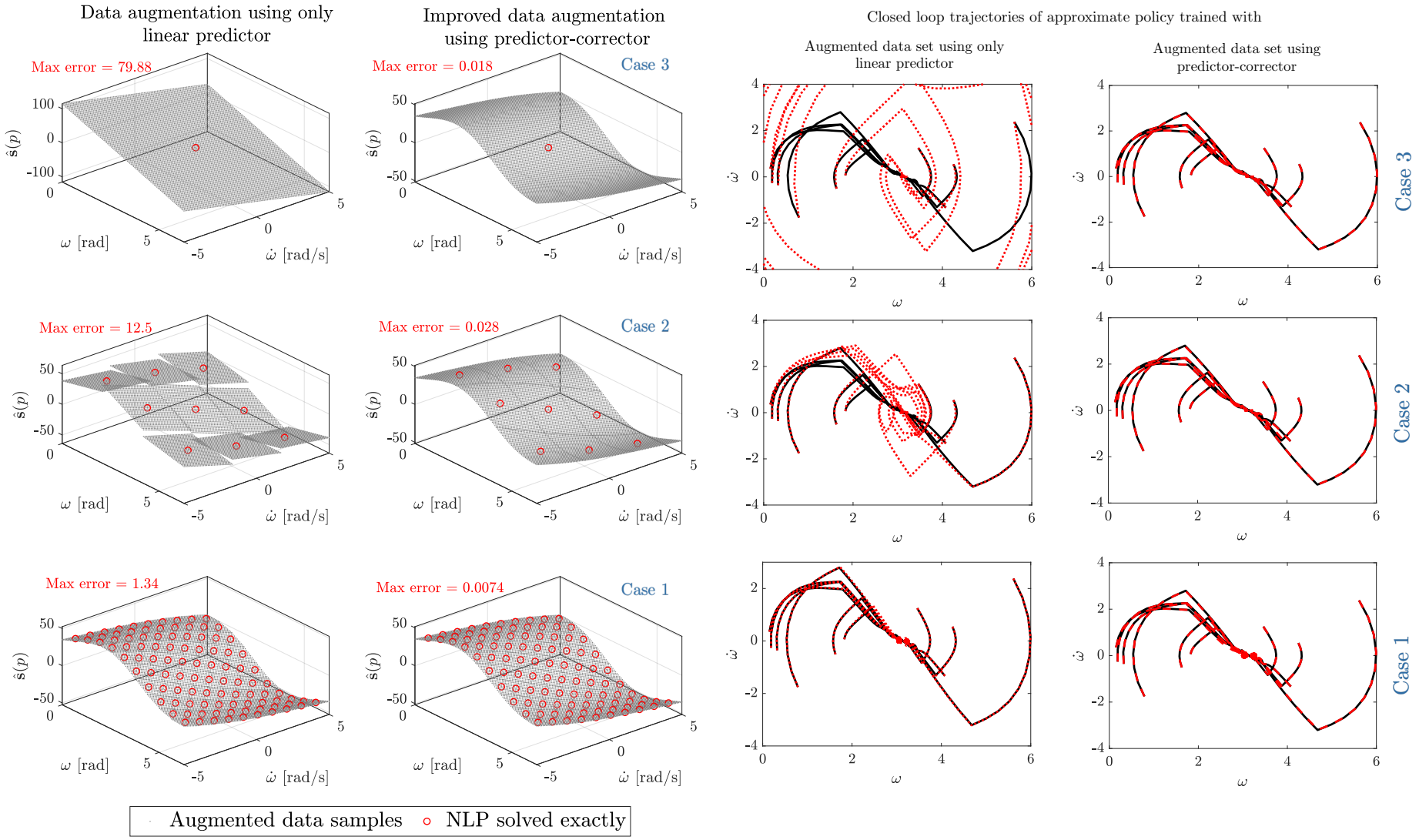
$$\|\pi(p; \hat{\theta}) - \pi^*(p)\| \leq \sup_{\mathbf{s}(p) \in \mathcal{S}(p)} \|\mathbf{s}(p) - \mathbf{s}^*(p)\|$$

in probability as $M \to \infty$ if θ_1 is a consistent estimator.

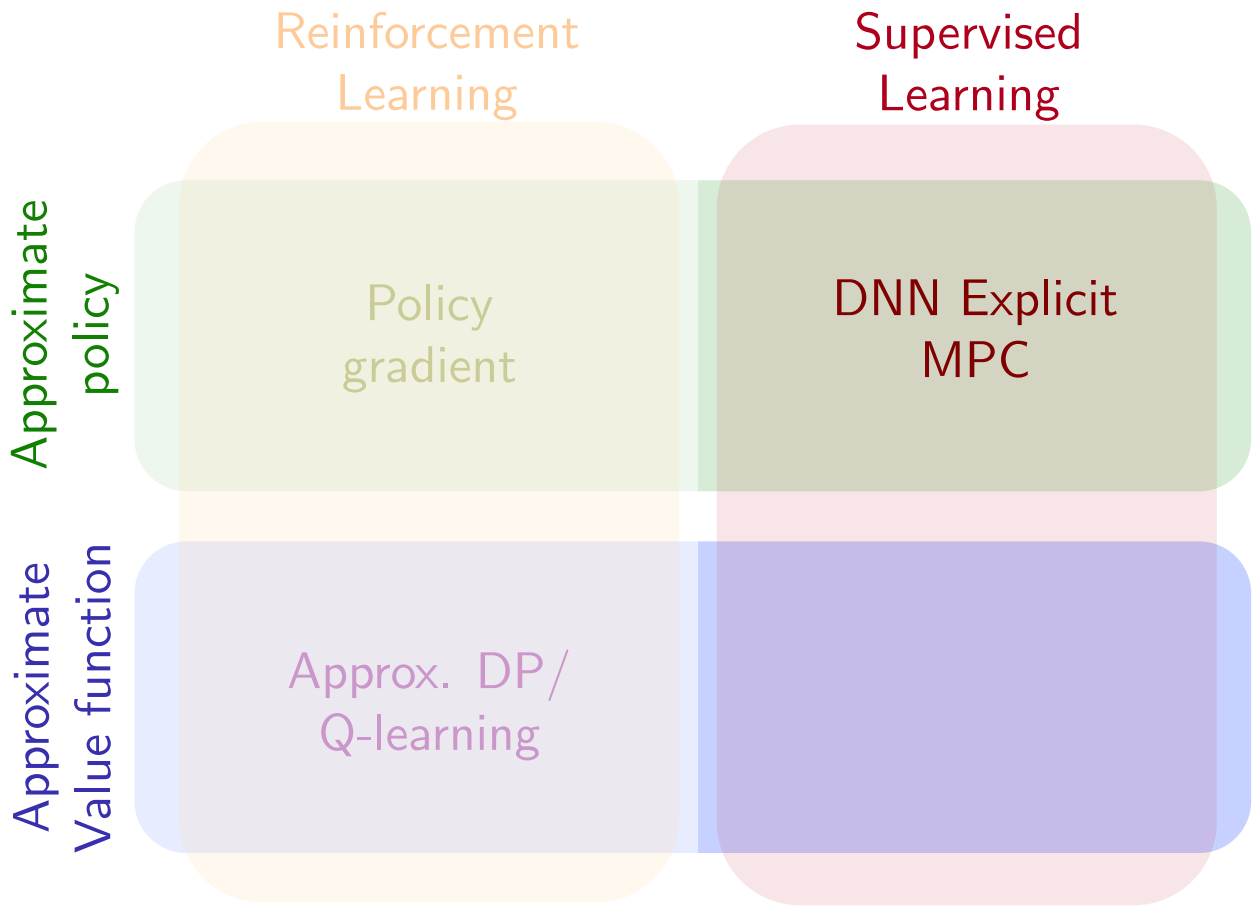




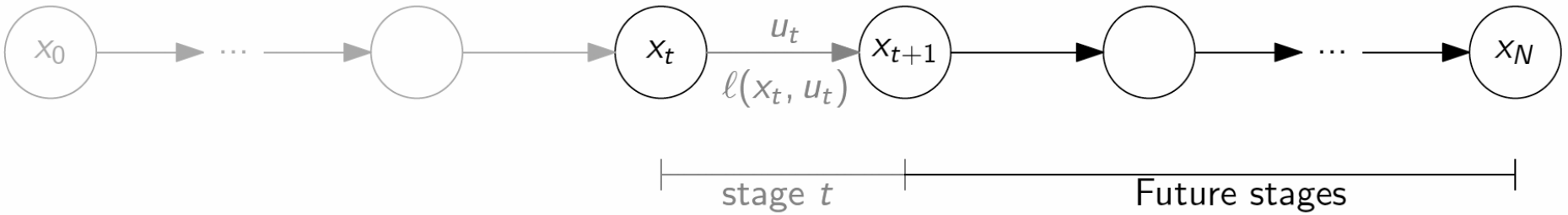
Sensitivity-based Data Augmentation



Krishnamoorthy, *IEEE L-CSS* 2024


Reinforcement Learning
Supervised Learning
Approximate policy
Approximate Value function
Policy gradient
DNN Explicit MPC
Approx. DP/ Q-learning

Sequential decision making

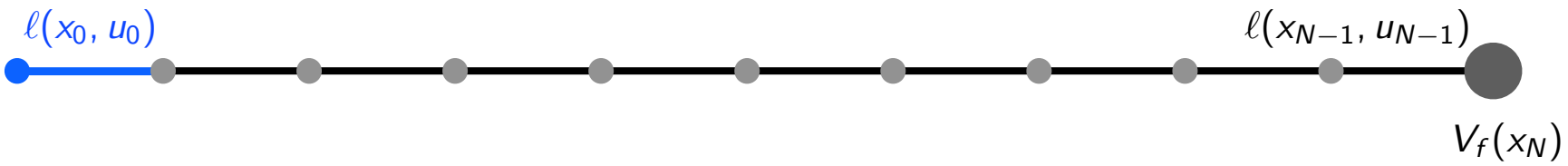


$$\pi^*(x) = \arg\min_{u \in \mathcal{U}} \ \ell(x, u) + J^*(f(x, u))$$

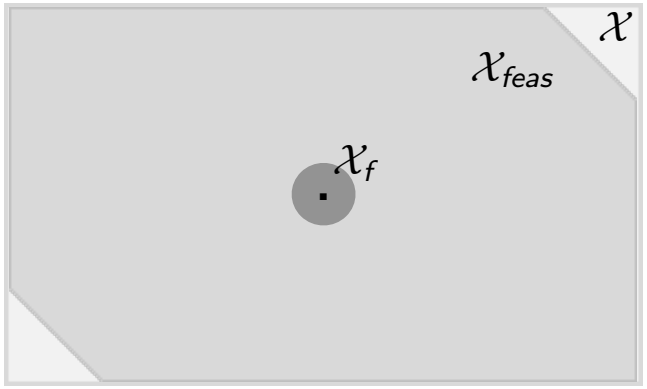
$$\approx$$

$$\pi^*(x) = \arg\min_{u \in \mathcal{U}} \ \ell(x, u) + \sum_{k=1}^{N-1} \ell(x_k, u_k) + V_f(x_N)$$

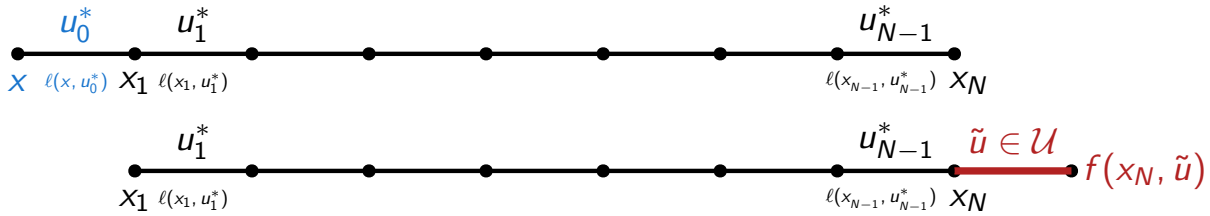
MPC - Horizon Rollout



$$
\begin{aligned}
V^*(x_t) = \min_{\mathbf{u}} \sum_{k=0}^{N-1} \ell(x_{k|t}, u_{k|t}) + V_f(x_{N|t}) \\
\text{s.t. } x_{k+1|t} = f(x_{k|t}, u_{k|t}) \\
x_{k|t} \in \mathcal{X}, u_{k|t} \in \mathcal{U} \\
x_N \in \mathcal{X}_f \\
x_{0|t} = x_t
\end{aligned}
$$



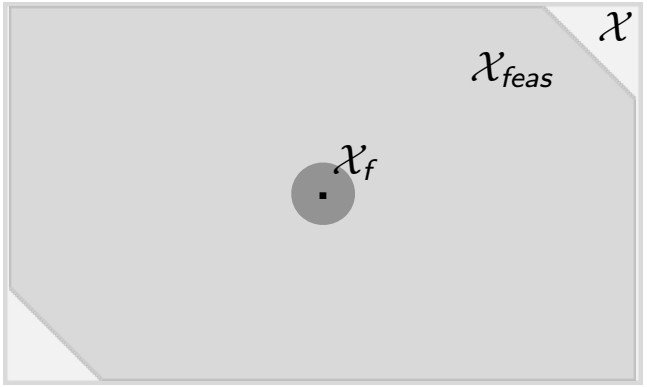
MPC - Horizon Rollout



Basic stability assumption

For all $x \in \mathcal{X}_f$, $\exists u \in \mathcal{U}$ satisfying

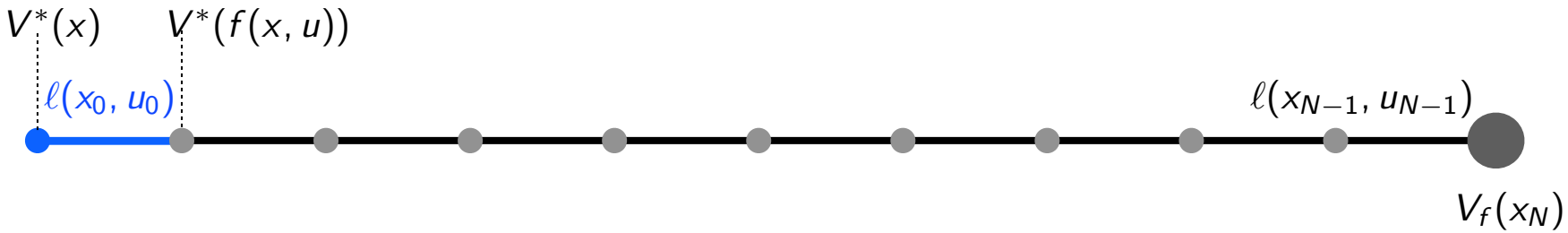
- $f(x, u) \in \mathcal{X}_f$
- $V_f(f(x, u)) - V_f(x) \leq -\ell(x, u)$,

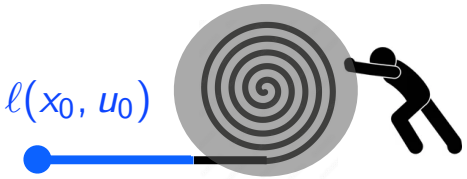


$$V_N(f(x, \pi(x))) - V_N(x) \leq -\ell(x, \pi(x)) \underbrace{-V_f(x_N) + \ell(x_N, \tilde{u}) + V_f(f(x_N, \tilde{u}))}_{\leq 0}$$

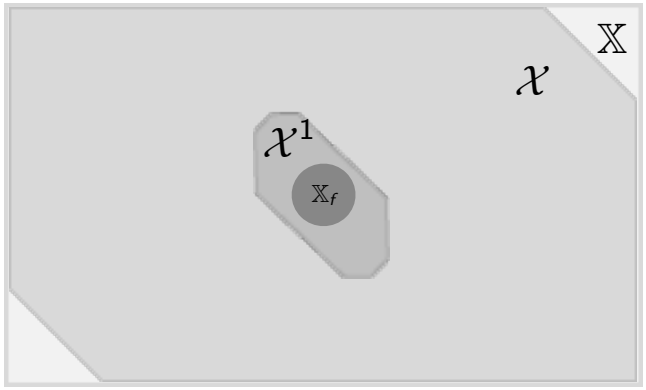
The origin is asymptotically stable under the MPC policy

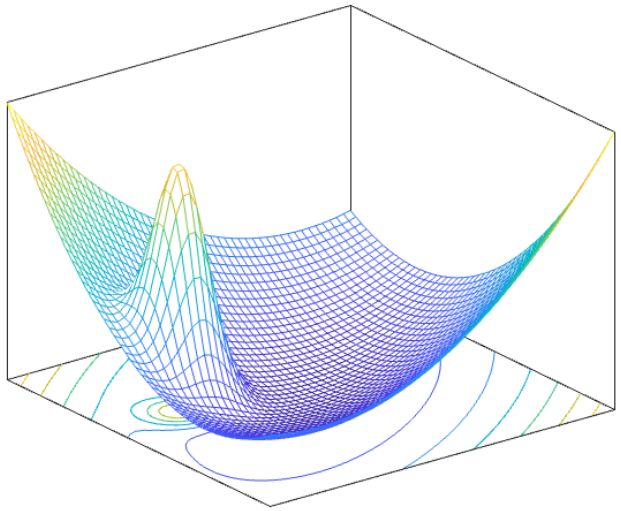
Rawlings, Mayne, Diehl, *MPC: Theory, computation, & design*, 2nd Ed., (2017)





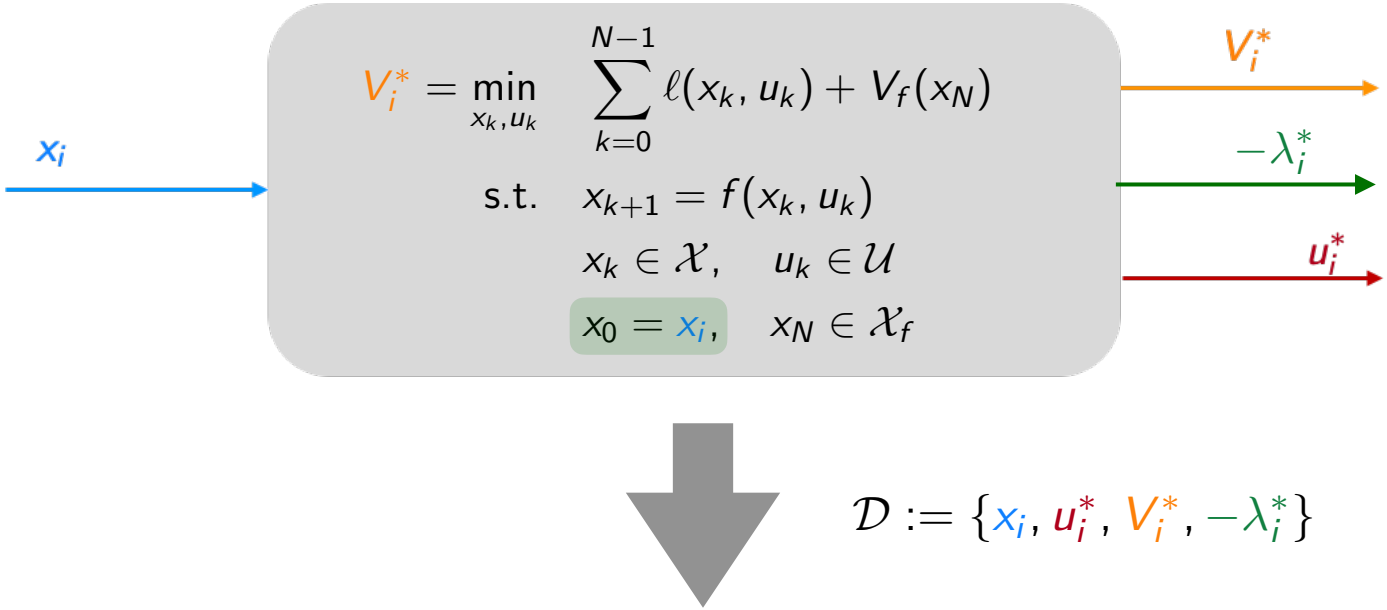
$$\min_u \ell(x_t, u) + V^*(f(x_t, u))$$
$$\text{s.t. } g(x_t, u) \leq 0$$





$$\mathcal{X}^1 = Pre(\mathbb{X}_f) := \{x \in \mathbb{X} \mid \exists u \in \mathbb{U},\ f(x,u) \in \mathbb{X}_f\}$$

Goal: Synthesize a terminal cost $\mathcal{V}_\theta(x)$



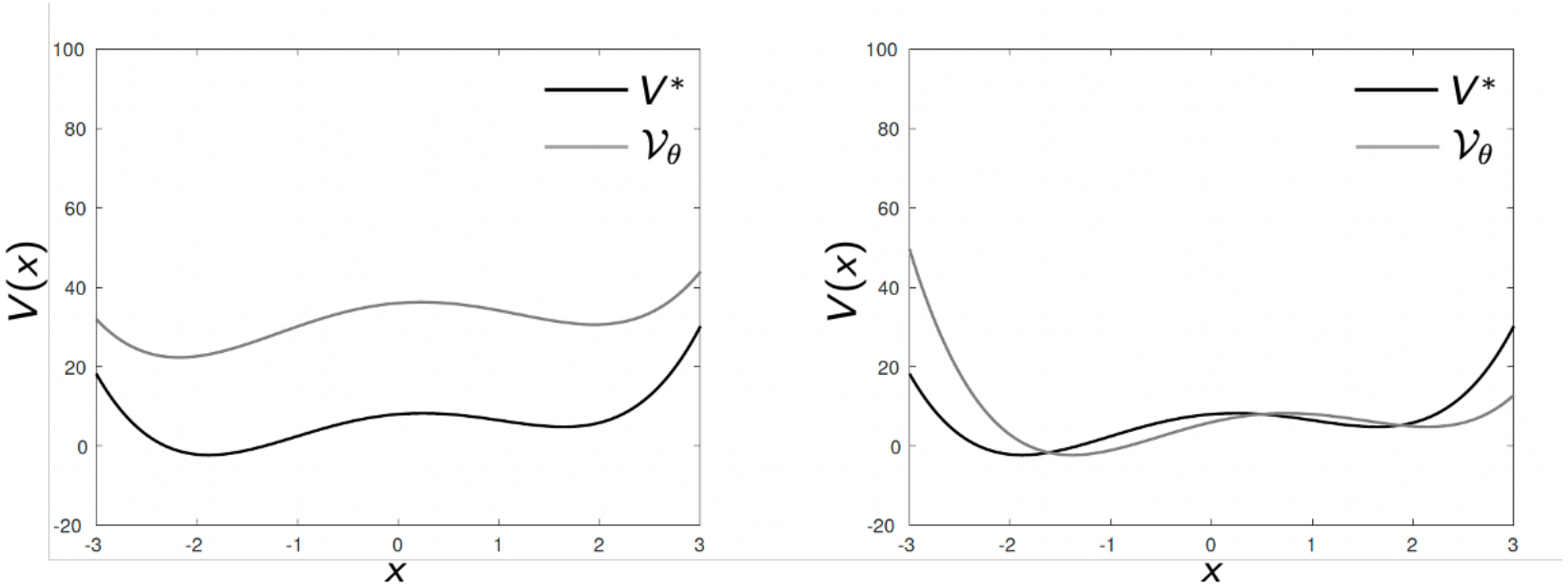
Myopic MPC

$$\begin{aligned} \min_{u \in \mathcal{U}} \quad & \ell(x, u) + \mathcal{V}_\theta(x^+) \\ \text{s.t.} \quad & x^+ = f(x, u) \in \mathcal{X} \end{aligned}$$

What constitutes a good approximation?

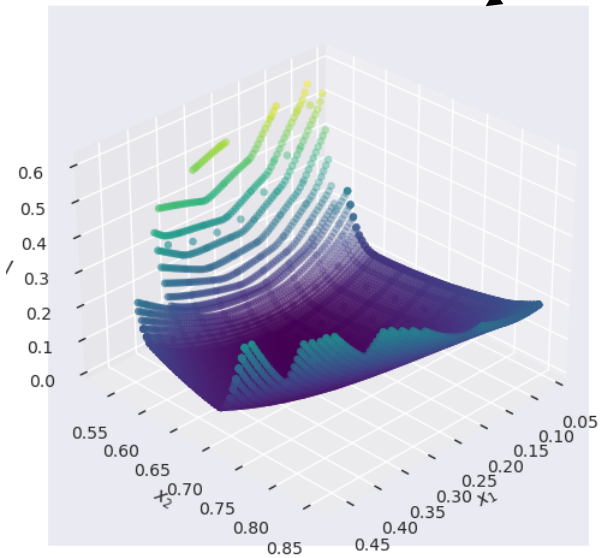
- $\mathcal{V}_\theta$ must be close enough to V_i^* for all i

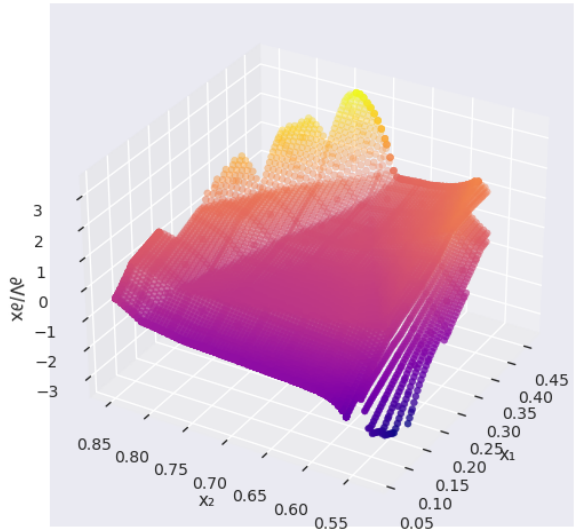
$$\hat{\theta} = \arg\min_\theta \sum_i \|\mathcal{V}_\theta(x_i) - V_i^*\|^2$$

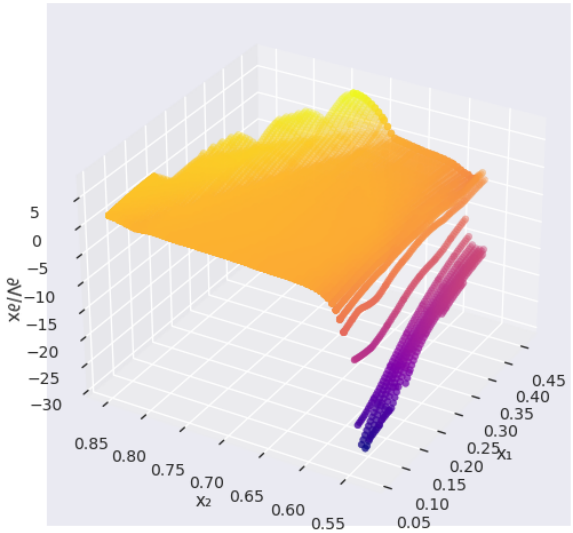


Learning terminal costs with Sobolev training

$$\min_{\theta} \sum_{i} \left[\|\mathcal{V}_\theta(x_i) - V_i^*\|^2 + \alpha \|\nabla_x \mathcal{V}_\theta(x_i) + \lambda_i^*\|^2 \right]$$





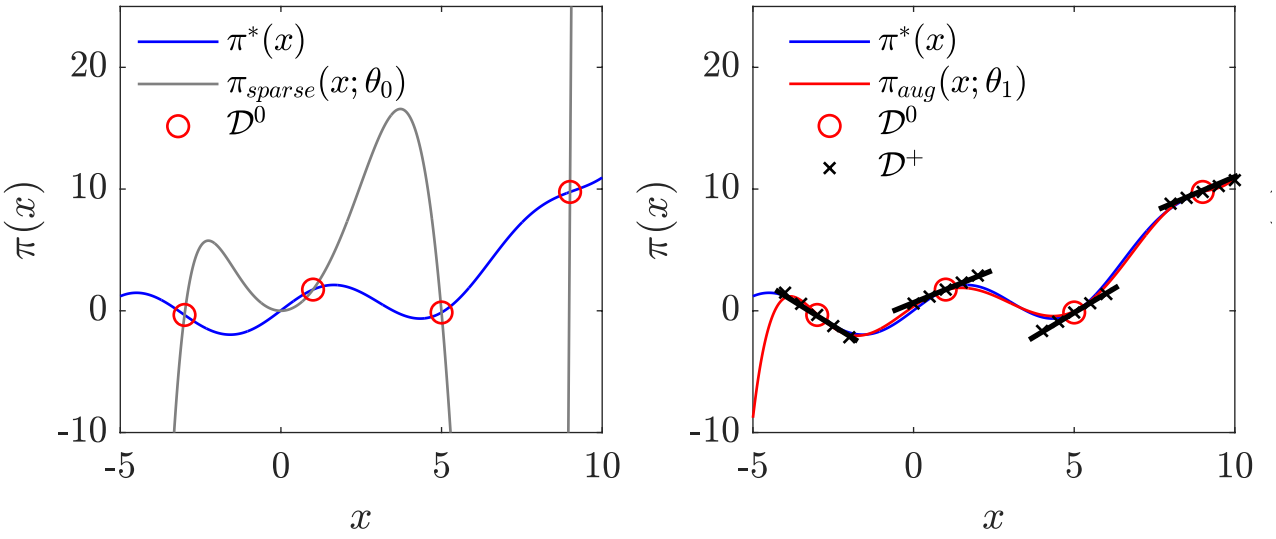


Vapnik-Chervonenkis (VC) dimension

VC-dimension measures how many data points the model can "shatter".

Shattering

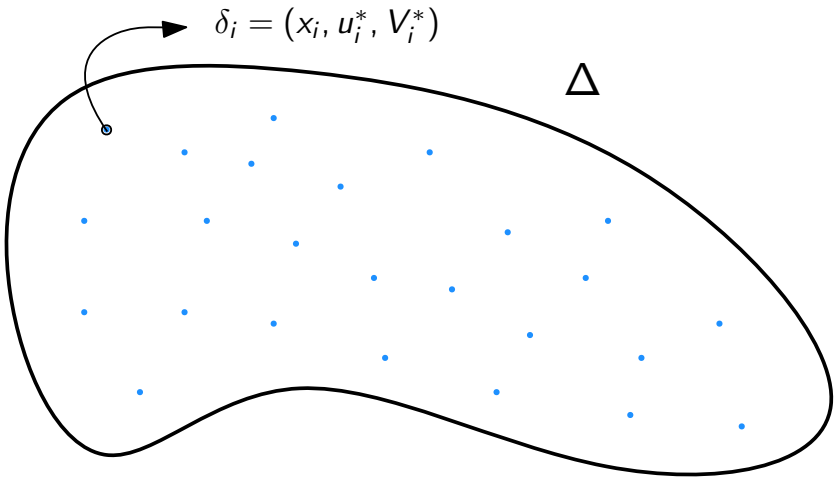
$\exists\, \theta$ for any give set of data, such that loss $\mathcal{L}(\theta) = 0$



The effective VC-dimension under Sobolev is reduced

Vidyasagar, *Learning and Generalization* (1997)

Myopic MPC: Guarantees



- Samples $\{x_i\}$ forms a ρ-net.

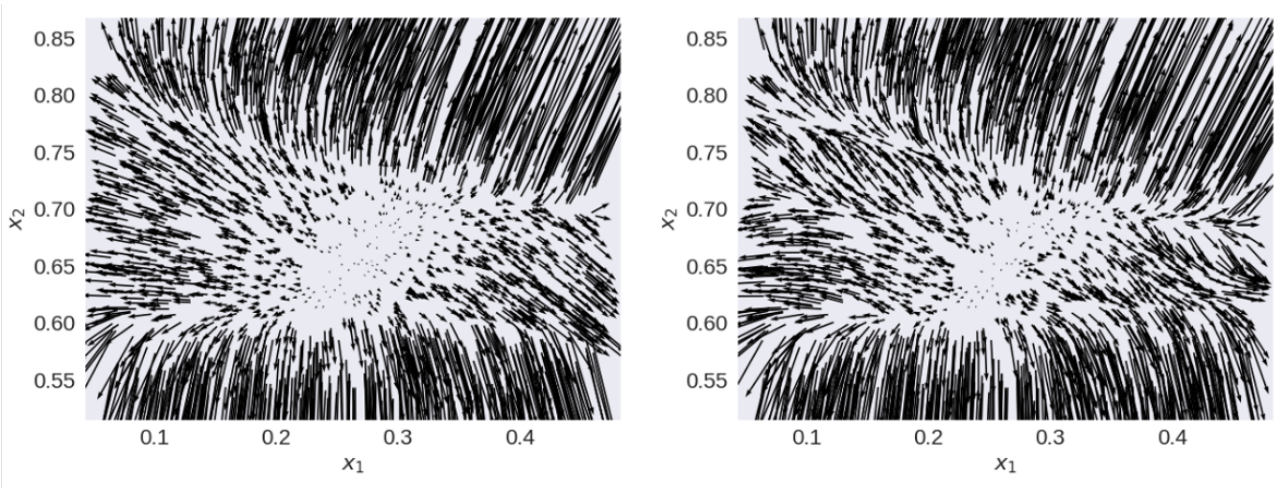
  $\exists\ i$ such that $\|x - x_i\| \leq \rho$ for all $x \in \mathcal{X}_{feas}$

- Pointwise bound at the finite samples

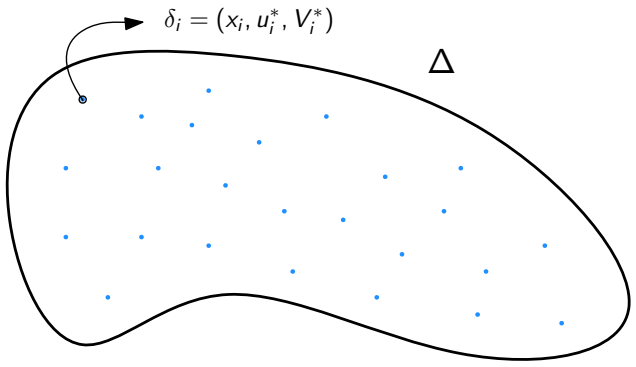
  $$\max_{1 \leq i \leq M} \|\nabla \mathcal{V}_\theta(x_i) - \nabla V^*(x_i)\| \leq d_M$$

- $\nabla \mathcal{V}(x)$ is Locally Lipschitz

$$\sup_{x \in \mathcal{X}_{feas}} \|\nabla \mathcal{V}_\theta(x) - \nabla V^*(x)\| \leq d_M + 2L\rho =: \varepsilon_g$$



Myopic MPC: Guarantees



$$\min_{\theta} \sum_{i} \left[\|\mathcal{V}_\theta(x_i) - V_i^*\|^2 + \alpha \|\nabla_x \mathcal{V}_\theta(x_i) + \lambda_i^*\|^2 \right]$$

sampling density affects ρ

gradient error term d_M

Boundedness of the policy

If LICQ and SOSC holds, the induced policy is uniformly close to the ideal policy

$$\|\pi_\theta(x) - \pi^*(x)\| \leq L_p(d_M + 2L\rho)$$

Boundedness of the closed-loop trajectories

If system dynamics are Lipschitz, the closed-loop trajectories are uniformly close

$$\|f(x, \pi_\theta(x)) - f(x, \pi^*(x))\| \leq L_f \|\pi_\theta(x) - \pi^*(x)\| \leq L_f L_p(d_M + 2L\rho)$$

Minten et al. (2026)

Myopic MPC: Guarantees

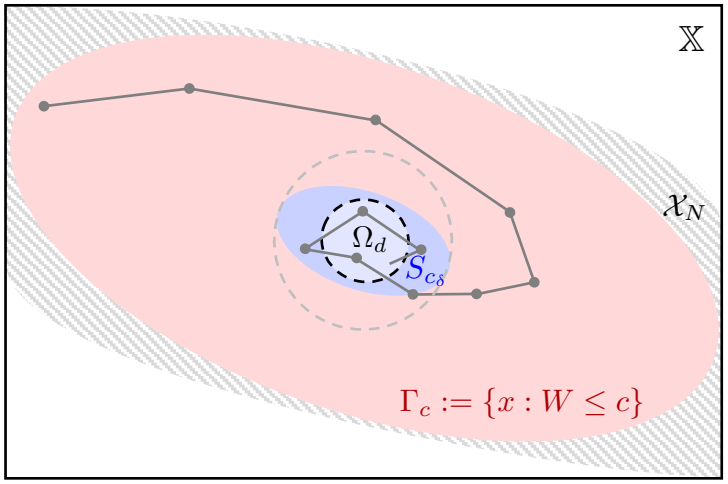
Practical stability

If $\exists$ a Lyapunov function $W : \mathcal{X}_{feas} \to \mathbb{R}_{\geq 0}$ such that $\alpha_1(\|x\|) \leq W(x) \leq \alpha_2(\|x\|)$, and $W\left(f(x, \pi^*(x))\right) - W(x) \leq -\alpha_3(\|x\|)$

$$W(f_\theta(x)) - W(x) \leq \delta - \alpha_3(\|x\|)$$

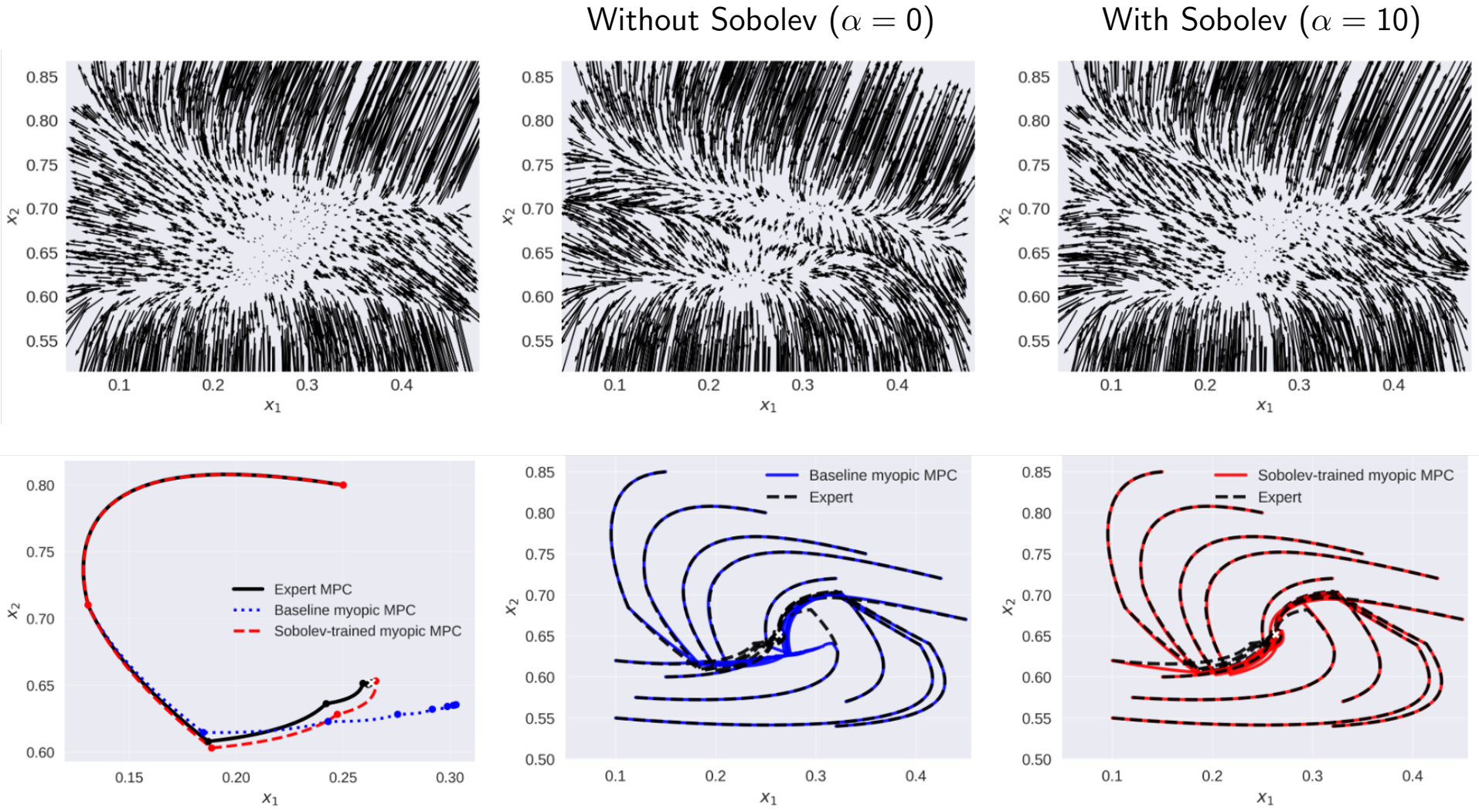
where $\delta := L_W L_f L_p(d_M + 2L\rho)$ and the trajectories converge to the residual set

$$\Omega_\delta := \{x \in \mathcal{X}_{feas} : \alpha_3(\|x\|) \leq \delta\}$$



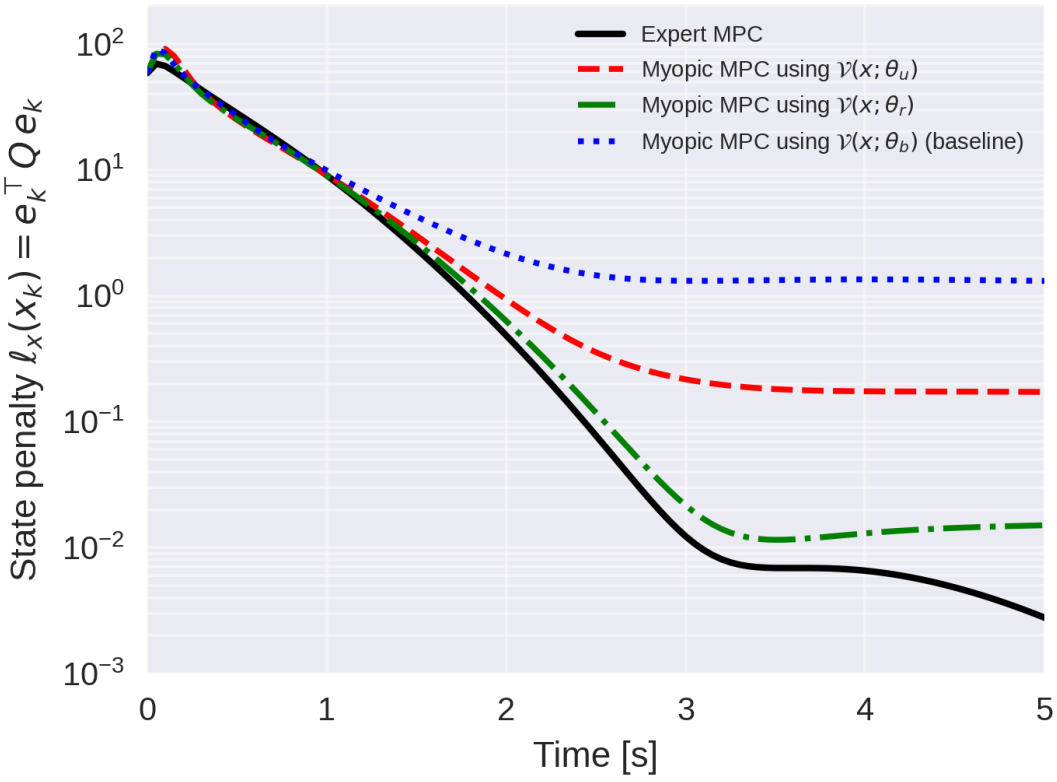
Minten et al. (2026)

Example: Benchmark CSTR



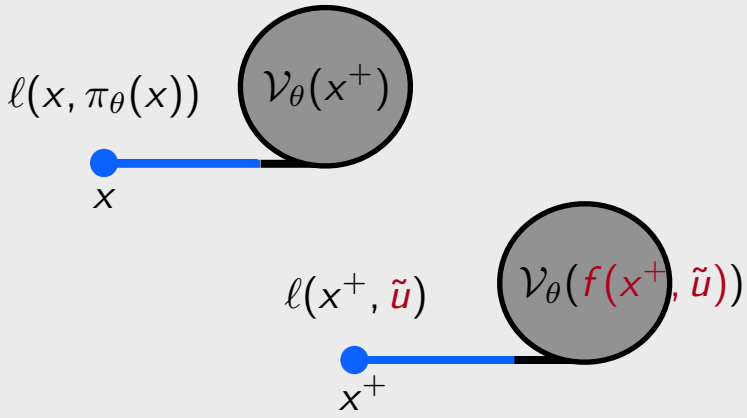
Minten et al. (2026)

Example: Quadcopter



MPC nominal stability

$$\begin{aligned} \min_{u \in \mathcal{U}} \quad & \ell(x, u) + \mathcal{V}_\theta(x^+) \\ \text{s.t.} \quad & x^+ = f(x, u) \in \mathcal{X} \end{aligned}$$



$$V_1(f(x, \pi_\theta(x))) \leq V_1(x^+, \tilde{u})$$

$$V_1(f(x, \pi(x))) - V_1(x) \leq -\ell(x, \pi_\theta(x)) \underbrace{-\mathcal{V}_\theta(x^+) + \ell(x^+, \tilde{u}) + \mathcal{V}_\theta(f(x^+, \tilde{u}))}_{\leq 0 \ \forall x \in \mathcal{X}_{feas}?}$$

What constitutes a good approximation?

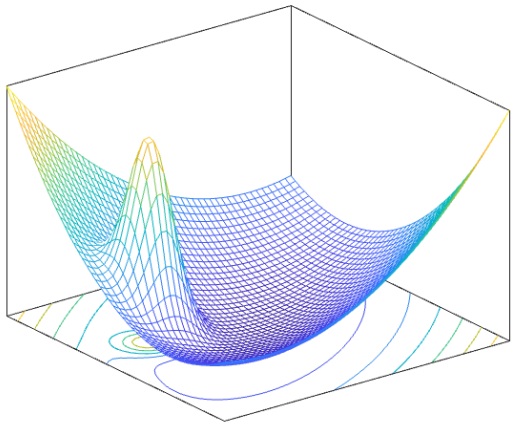
Desirable property

- Upon applying optimal u_i^* at x_i,
- cost-to-go from the successor state $\mathcal{V}_\theta(f(x_i, u_i^*))$ must be lower than $\mathcal{V}_\theta(x_i)$

Learn with descent property constraint

$$\hat{\theta} = \arg\min_{\theta} \quad \sum_i \|\mathcal{V}_\theta(x_i) - V_i^*\|^2$$
$$\text{s.t.} \quad \mathcal{V}_\theta(f(x_i, u_i^*)) - \mathcal{V}_\theta(x_i) \leq -\ell(x_i, u_i^*), \quad \forall i$$

- Special case: $\mathcal{V}_\theta(x) := x^\top P x$, with $P = L^\top L$



Abdufattokhov et al. *IEEE CDC* (2021);

Learn terminal cost with descent property constraint

Semi-infinite synthesis problem

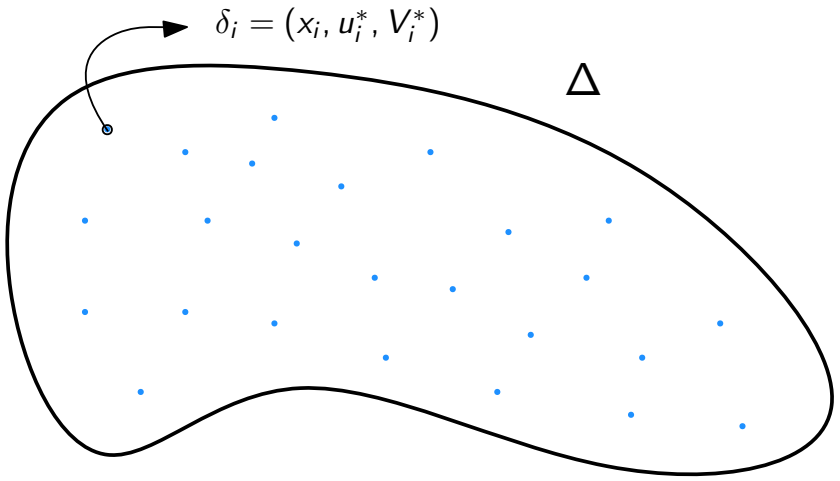
$$\min_{\theta \in \mathbb{R}^d} \quad \sum_i \mathcal{G}\left(\mathcal{V}_\theta(x), V\right)$$

$$\text{s.t.} \quad \mathcal{V}_\theta\left(f(x,u)\right) - \mathcal{V}_\theta(x) \leq -\ell(x,u), \quad \forall x \in \mathcal{X}_{feas}$$

Scenario problem

$$SP_M : \min_{\theta \in \mathbb{R}^d} \sum_i \mathcal{G}\left(\mathcal{V}_\theta(x_i), V_i^*\right)$$

$$\text{s.t.} \quad \mathcal{V}_\theta\left(f(x_i, u_i^*)\right) - \mathcal{V}_\theta(x_i) \leq -\ell(x_i, u_i^*), \quad \forall i = 1, \ldots, M$$

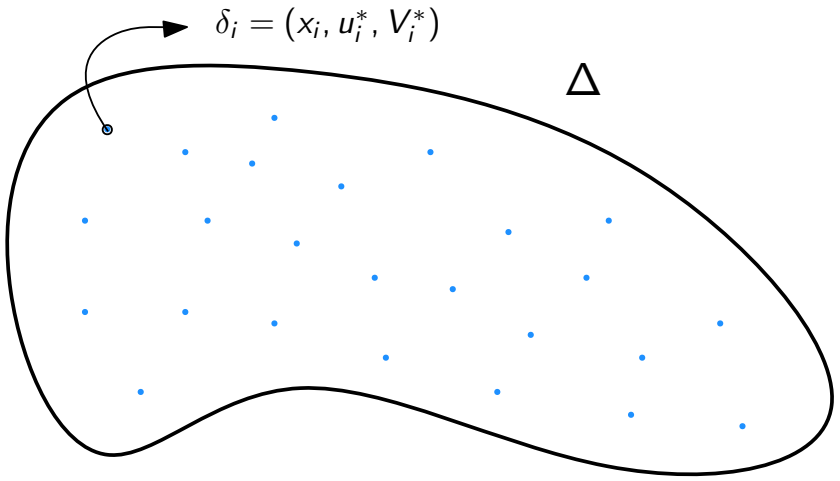


- For all $\mathcal{D} = \{\delta_i\} \in \Delta$, we have $\theta \in \Theta_\delta$
- What can we say about constraint satisfaction for any $\delta \in \Delta$?

Learn terminal cost with descent property constraint

Scenario problem

$$
\begin{aligned}
SP_M \ : \ & \min_{\theta \in \mathbb{R}^d} c^\top \theta, \\
\text{s.t. } & \theta \in \Theta_{\delta^{(i)}}, \\
& \forall \delta^{(i)}, i = 1, \dots, M,
\end{aligned}
$$



- For all $\mathcal{D} = \{\delta_i\} \in \Delta$, we have $\theta \in \Theta_\delta$
- What can we say about constraint satisfaction for any $\delta \in \Delta$?

Learn terminal cost with descent property constraint

$$SP_M : \min_{\theta \in \mathbb{R}^d} c^\top \theta,$$
$$\text{s.t. } \theta \in \Theta_{\delta^{(i)}},$$
$$\forall \delta^{(i)}, i = 1, \dots, M,$$

- SP_M is convex in θ
- $\{\delta_i\}$ are i.i.d samples from Δ
- SP_M is feasible and solution θ^* exists and is unique

Violation probability

For a given $\theta \in \Theta$, violation probability is defined as

$$\mathcal{P}(\theta) = \mathbb{P}\{\delta \in \Delta : \theta \notin \Theta_\delta\}$$

Generalization theorem of the Scenario Problem

Under these assumptions the following holds

$$\mathbb{P}^M \left\{\mathcal{P}\left(\theta_M^*\right) > \epsilon\right\} \leq \sum_{i=0}^{d-1} \binom{M}{i} \epsilon^i (1-\epsilon)^{M-i}$$

Campi & Garatti, *SIAM J. on Opt.* (2008)

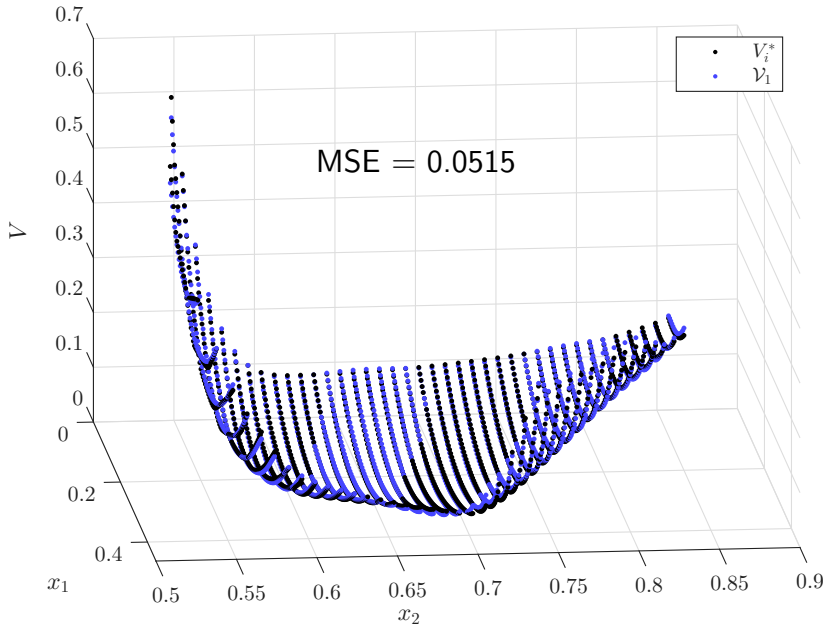
Probabilisitic certificate for the violation of the descent property

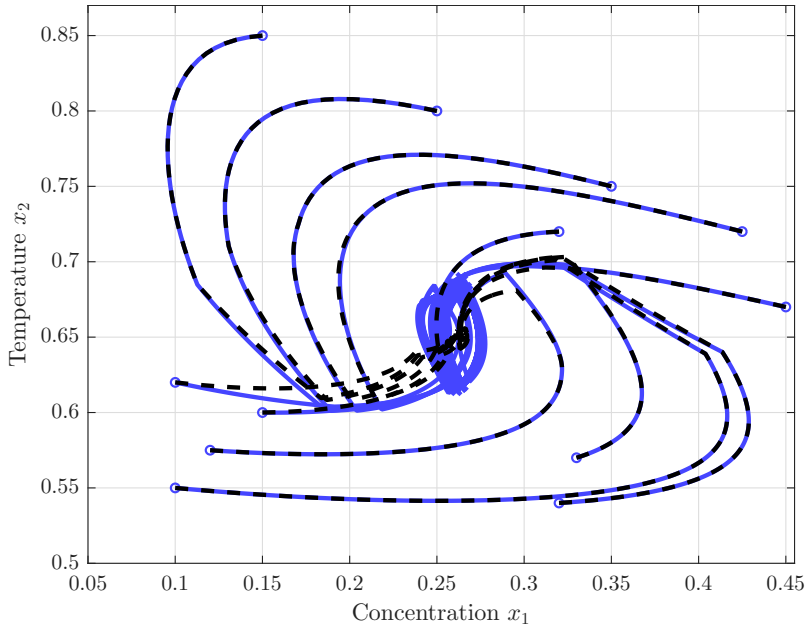
$$\mathbb{P}\{\mathcal{V}(f(x, \kappa_N(x)); \theta_M^*) - \mathcal{V}(x; \theta_M^*) \leq -\ell(x, \kappa_N(x))\} > \epsilon$$

Baltussen et al. *IEEE CDC* (2025)

Example: Benchmark CSTR

$$\min_{\theta} \quad \sum_{i} \|\mathcal{V}_\theta(x_i) - V_i^*\|^2$$

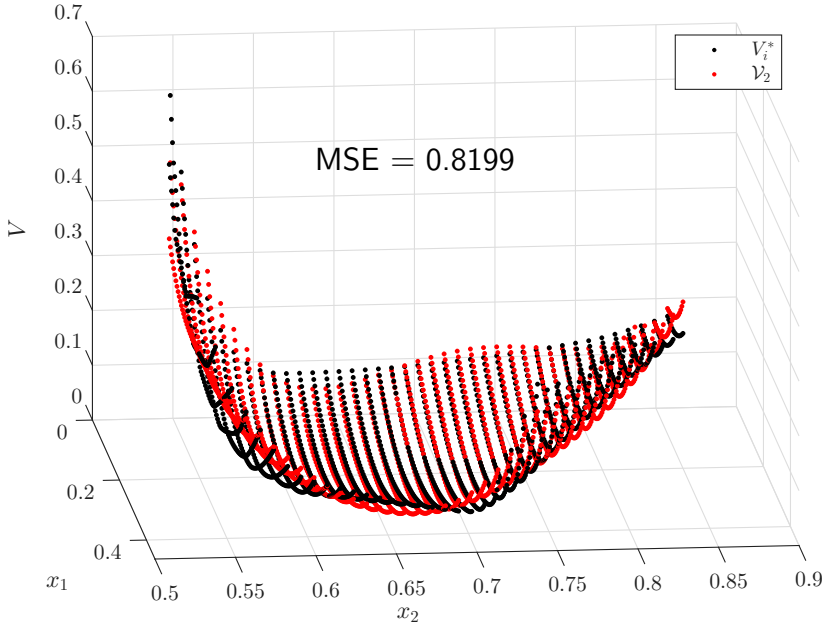


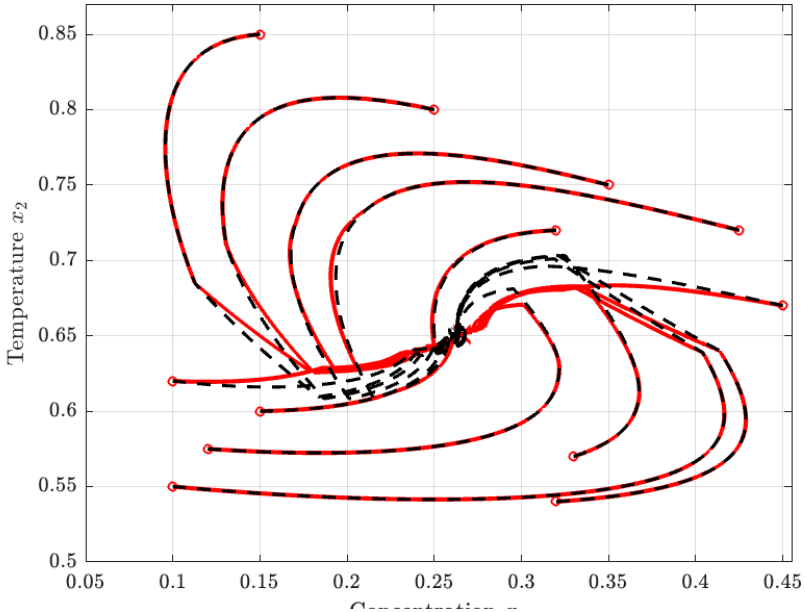


Example: Benchmark CSTR

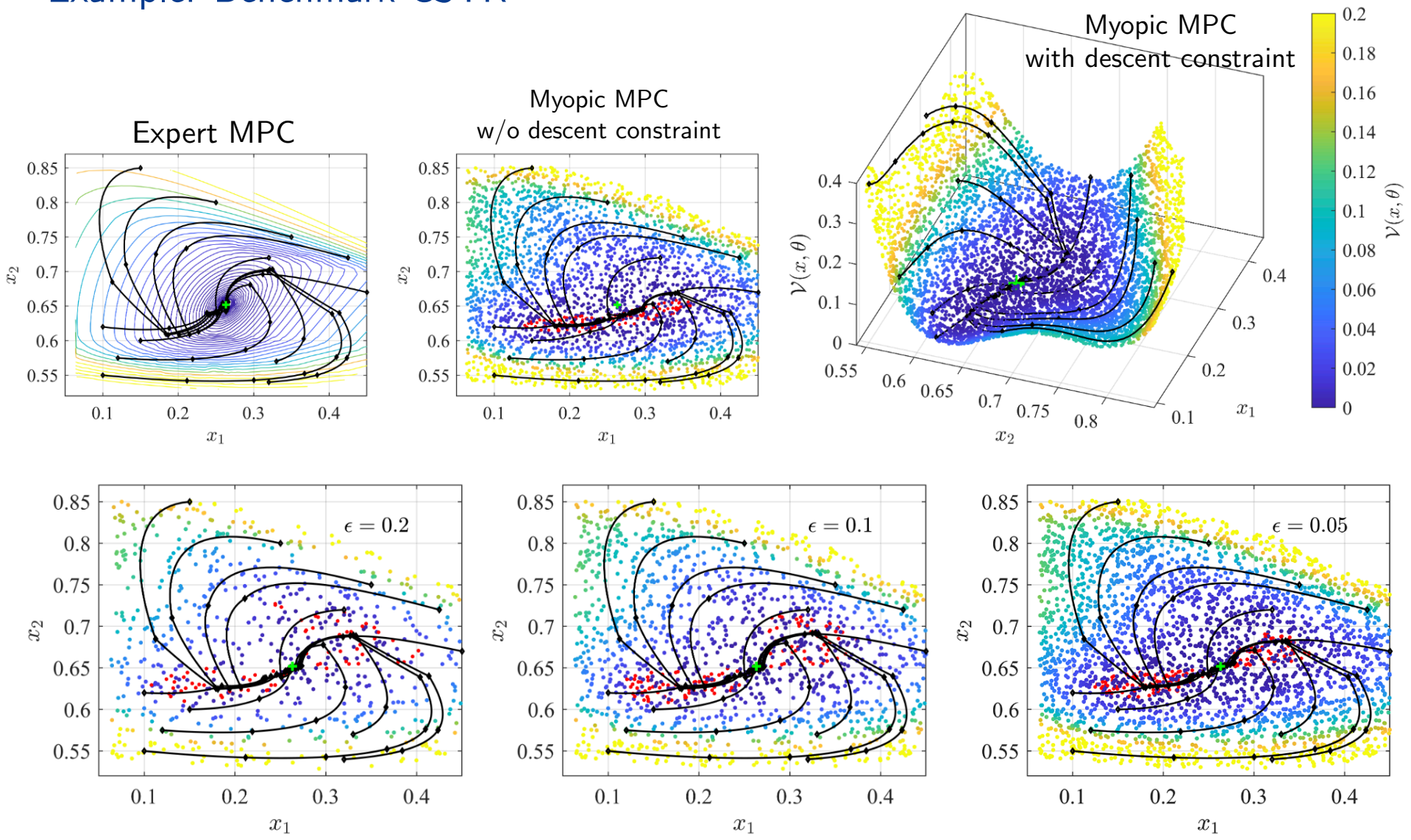
$$\min_{\theta} \quad \sum_i \|\mathcal{V}_\theta(x_i) - V_i^*\|^2$$

$$\text{s.t.} \quad \mathcal{V}_\theta(f(x_i, u_i^*)) - \mathcal{V}_\theta(x_i) \leq -\ell(x_i, u_i^*)$$





Example: Benchmark CSTR

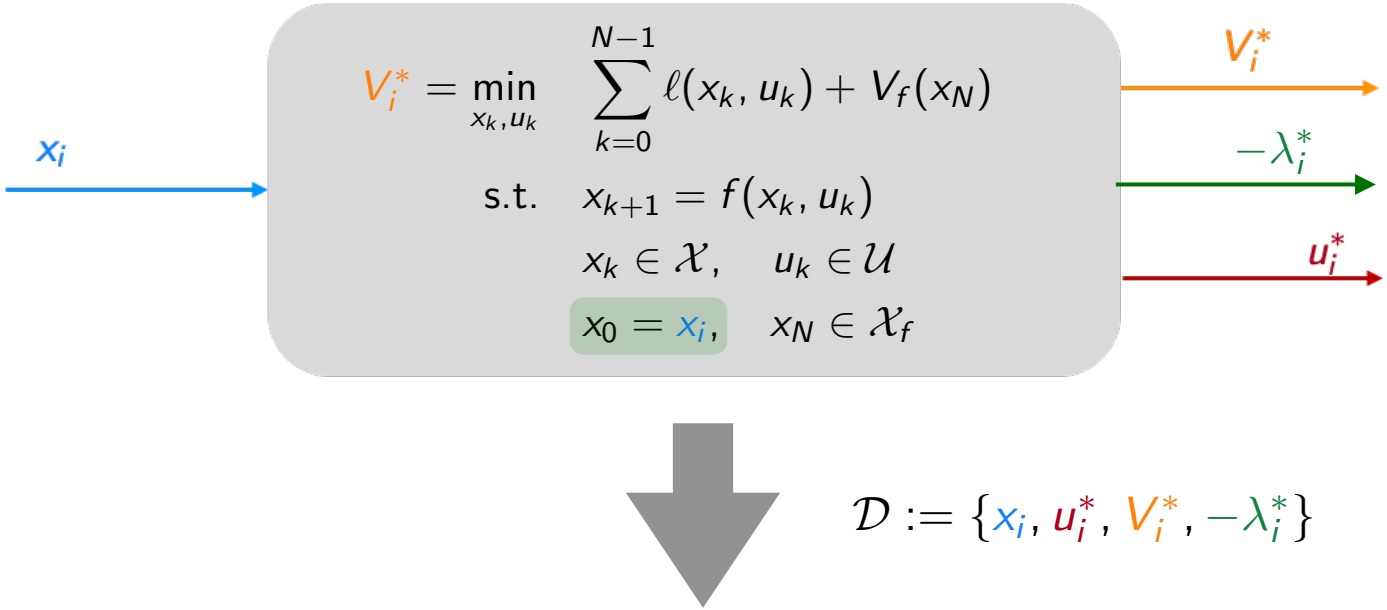


(a) Training points $M = 683$, Empirical violation $\tilde{\epsilon} = 0.1091$.

(b) Training points $M = 1403$, Empirical violation $\tilde{\epsilon} = 0.0869$.

(c) Training points $M = 2842$, Empirical violation $\tilde{\epsilon} = 0.0483$.

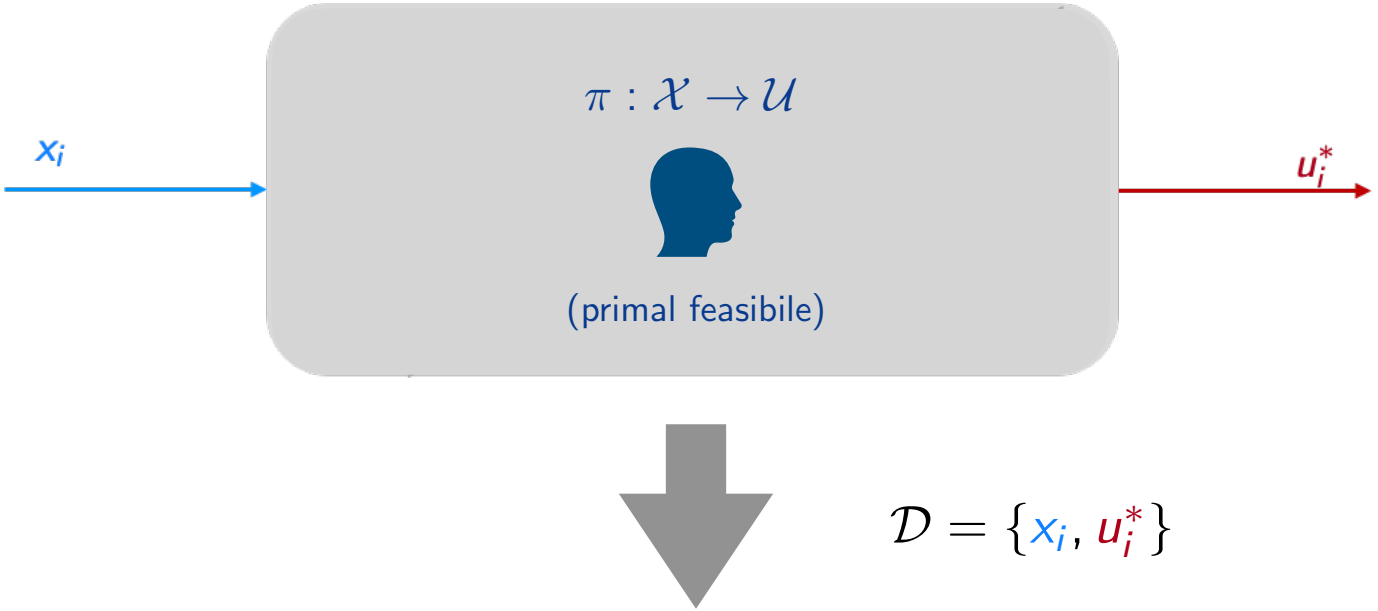
Goal: Synthesize a terminal cost $\mathcal{V}_\theta(x)$



Myopic MPC

$$\begin{aligned} \min_{u \in \mathcal{U}} \quad & \ell(x, u) + \mathcal{V}_\theta(x^+) \\ \text{s.t.} \quad & x^+ = f(x, u) \in \mathcal{X} \end{aligned}$$

Goal: Synthesize terminal cost from $\mathcal{D} = \{x_i, u_i^*\}$



Myopic MPC

$$\begin{aligned} \min_{u \in \mathcal{U}} \quad & \ell(x, u) + \mathcal{V}_\theta(x^+) \\ \text{s.t.} \quad & x^+ = f(x, u) \in \mathcal{X} \end{aligned}$$

Learning terminal cost from expert demonstrations $\mathcal{D} = \{x_i, u_i^*\}$

Myopic MPC

$$\begin{aligned} \min_{u} \quad & \ell(x, u) + \mathcal{V}_\theta(f(x, u)) \\ \text{s.t.} \quad & g(x, u) \leq 0 \end{aligned}$$

$$\mathcal{L}(x, u, \lambda) := \ell(x, u) + \mathcal{V}_\theta(f(x, u)) + \lambda^\top g(x, u) \qquad \text{(Lagrangian)}$$

KKT conditions of the myopic MPC

$$\begin{aligned} \nabla_u \ell(x, u) + \nabla_u f(x, u)^\top \nabla_x \mathcal{V}_\theta(f(x, u)) + \nabla_u g(x, u)^\top \lambda = 0 \quad & \text{(stationarity)} \\ \lambda \circ g(x, u) = 0 \quad & \text{(complementary slackness)} \\ \lambda \geq 0 \quad & \text{(Dual feasibility)} \end{aligned}$$

For what values of θ, do the data points $\mathcal{D} = \{(x_i, u_i^*)\}$,
satisfy the KKT conditions of the myopic MPC?

Learning terminal cost from expert demonstrations $\mathcal{D} = \{x_i, u_i^*\}$

For what values of θ, do the data points $\mathcal{D} = \{(x_i, u_i^*)\}$,
satisfy the KKT conditions of the myopic MPC?

KKT residuals

$$r_{stat}^{(i)}(\theta, \lambda_i) := \nabla_u \ell(x_i, u_i^*) + \nabla_u f(x_i, u_i^*)^\top \nabla_x \mathcal{V}_\theta(f(x_i, u_i^*)) + \nabla_u g(x_i, u_i^*)^\top \lambda_i$$
$$r_{comp}^{(i)}(\theta, \lambda_i) := \lambda_i \circ g(x_i, u_i^*)$$

KKT residual minimization

$$\min_{\theta, \{\lambda_i\}} \quad \sum_i \|r_{stat}^{(i)}(\theta, \lambda_i)\|^2 + \|r_{comp}^{(i)}(\theta, \lambda_i)\|^2 \qquad \text{(Stationarity \& complementary slackness)}$$

$$\text{s.t.} \quad \lambda_i \geq 0, \quad \forall i \qquad \text{(Dual feasibility)}$$

$$\mathcal{V}_\theta(f(x_i, u_i^*)) - \mathcal{V}_\theta(x_i) \leq -\ell(x_i, u_i^*), \quad \forall i \qquad \text{(descent property constraint)}$$

Keshavraz, Wang, Boyd, *IEEE MCSC* (2011)

KKT residual minimization for mixed-integer MPC

- Formulate myopic MPC with integer decisions
- Relax integer requirements
- Evaluate KKT residuals of the relaxed problem at finite samples $\mathcal{D} = \{x_i, u_i^*\}$
- Synthesize terminal cost $\mathcal{V}_\theta(x)$ by minimizing KKT residuals
- Use this in the myopic MPC with integer decisions

Myopic MPC

$$\begin{aligned} \min_{u} \quad & \ell(x, u) + \mathcal{V}_\theta(f(x, u)) \\ \text{s.t.} \quad & g(x, u) \leq 0 \end{aligned}$$

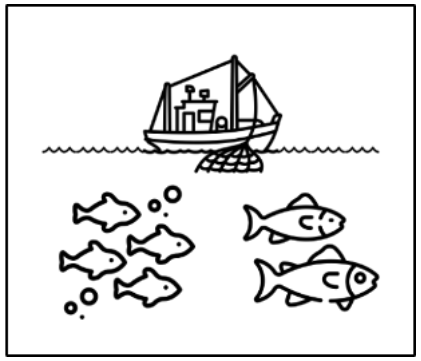
KKT residual minimization

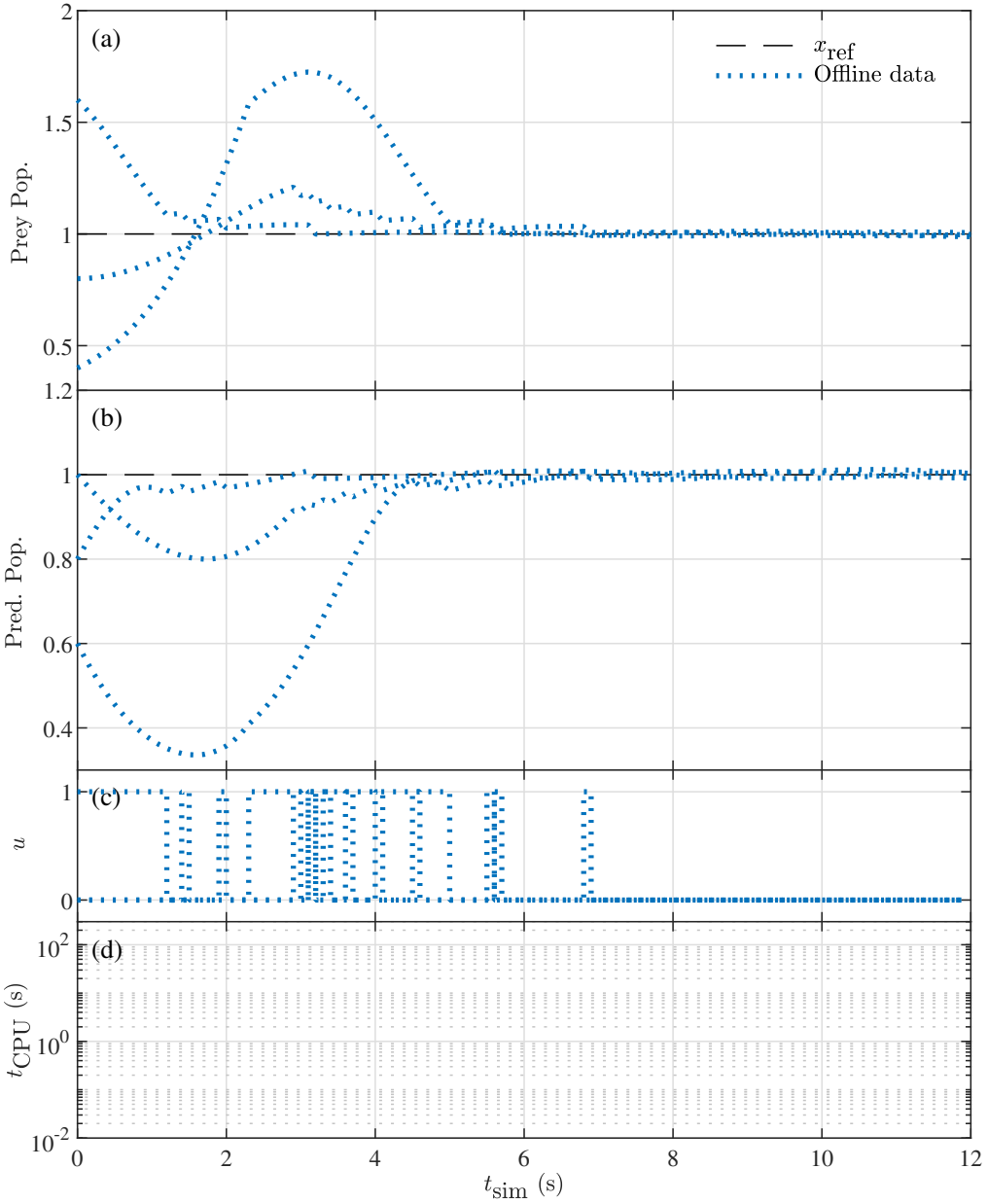
$$\begin{aligned} \min_{\theta, \{\lambda_i\}} \quad & \sum_i \|r_{stat}^{(i)}(\theta, \lambda_i)\|^2 + \|r_{comp}^{(i)}(\theta, \lambda_i)\|^2 && \text{(Stationarity \& complementary slackness)} \\ \text{s.t.} \quad & \lambda_i \geq 0, \quad \forall i && \text{(Dual feasibility)} \\ & \mathcal{V}_\theta(f(x_i, u_i^*)) - \mathcal{V}_\theta(x_i) \leq -\ell(x_i, u_i^*), \quad \forall i && \text{(descent property constraint)} \end{aligned}$$

Orrico et al. (2026)

Mixed-integer Myopic MPC using KKT residual minimzation

Lotka-Volterra problem

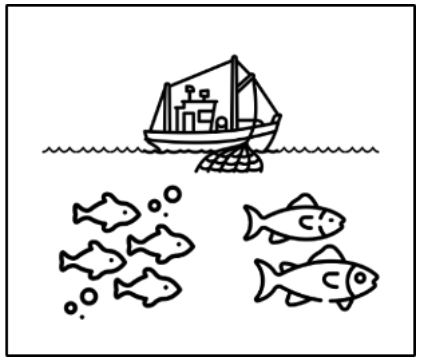


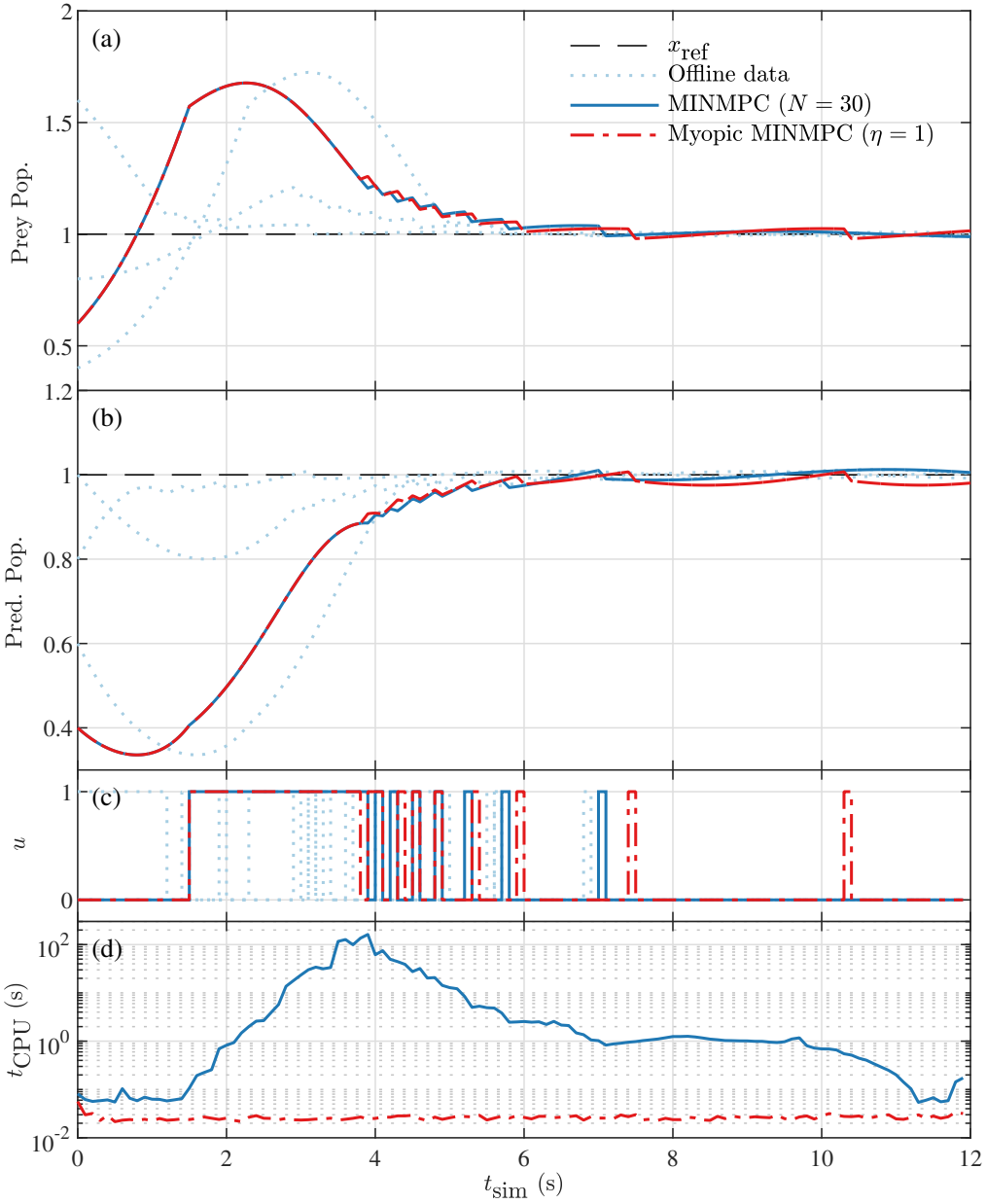


Orrico et al. (2026)

Mixed-integer Myopic MPC using KKT residual minimzation

Lotka-Volterra problem

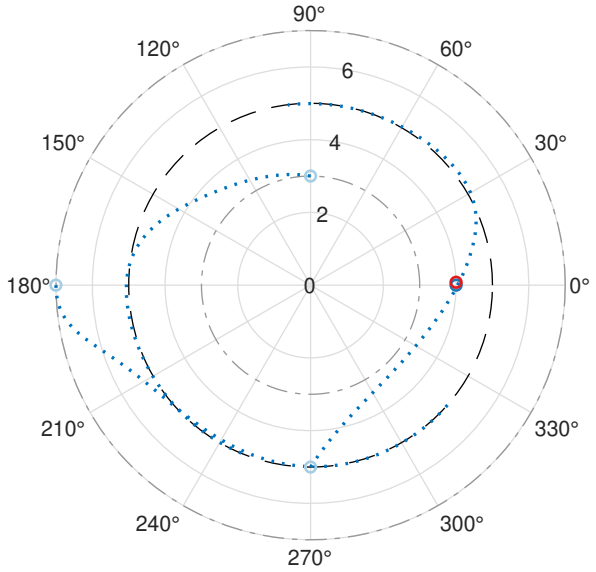


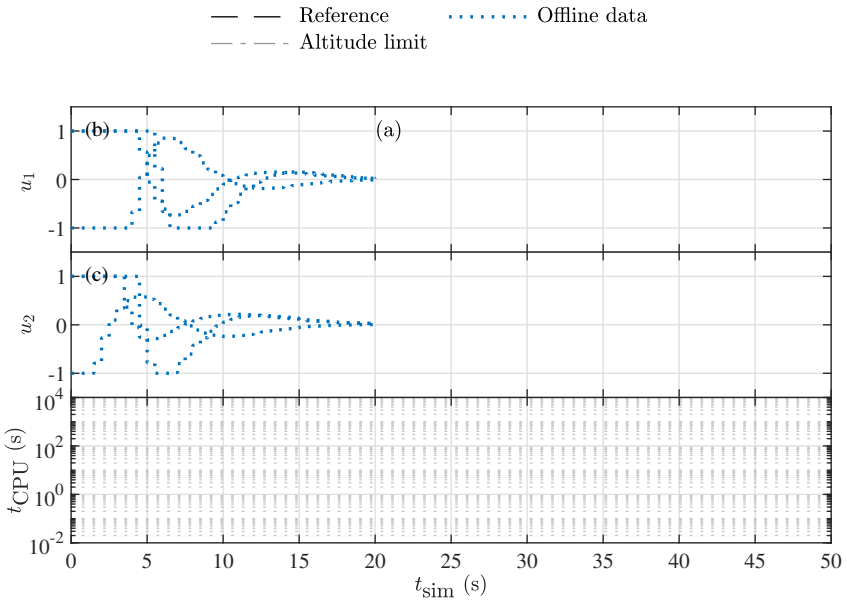


Orrico et al. (2026)

Mixed-integer Myopic MPC using KKT residual minimzation

Satellite in orbit

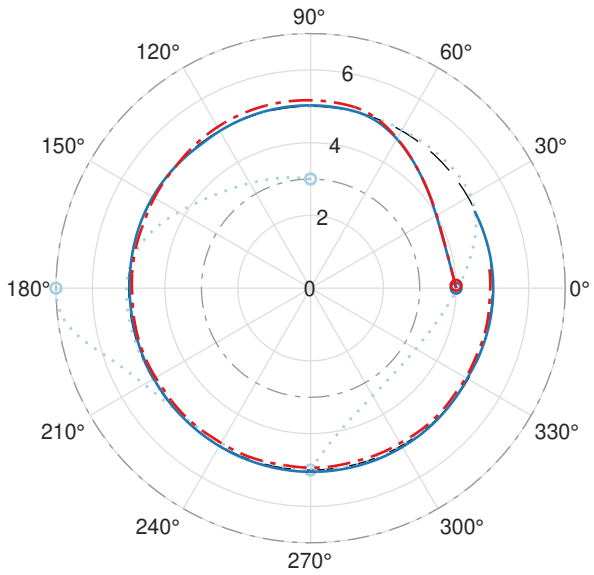


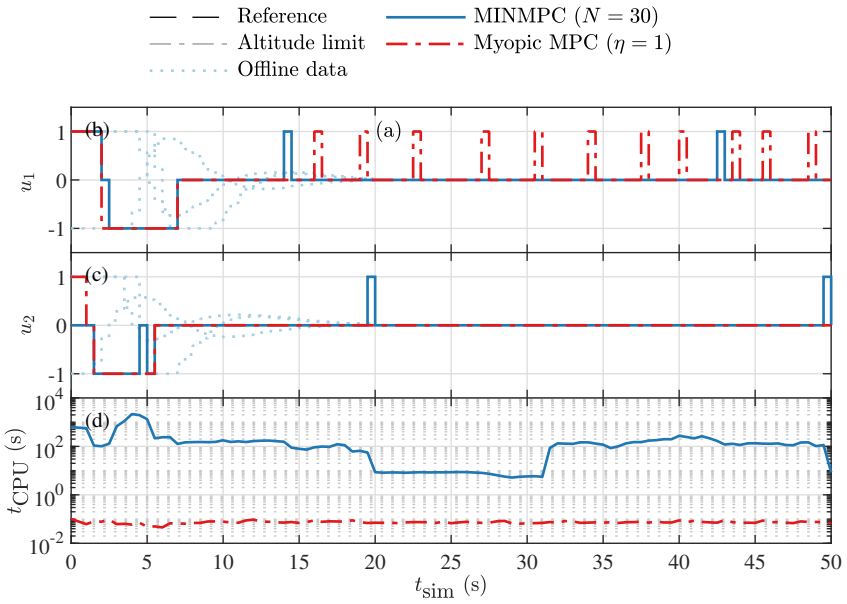


Orrico et al. (2026)

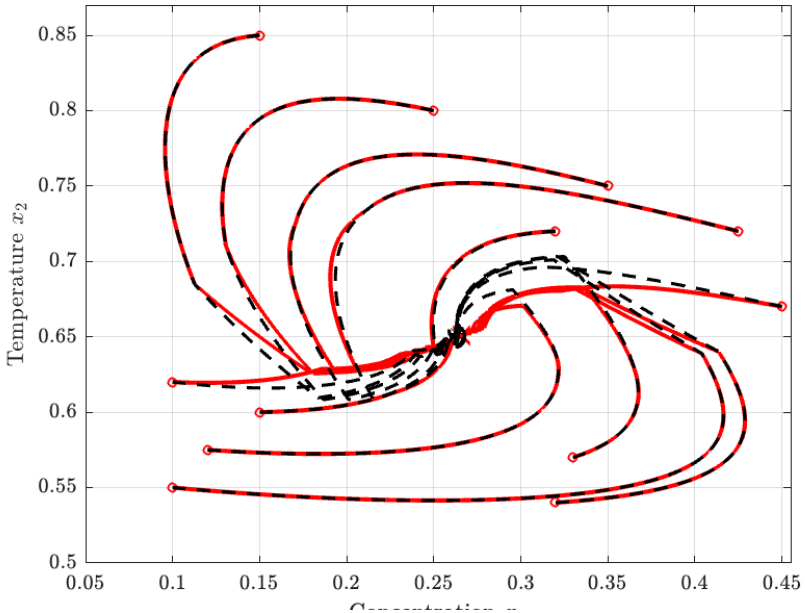
Mixed-integer Myopic MPC using KKT residual minimzation

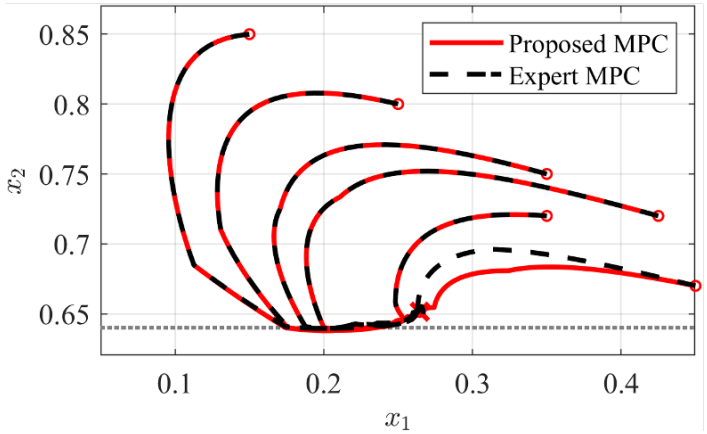
Satellite in orbit





Orrico et al. (2026)

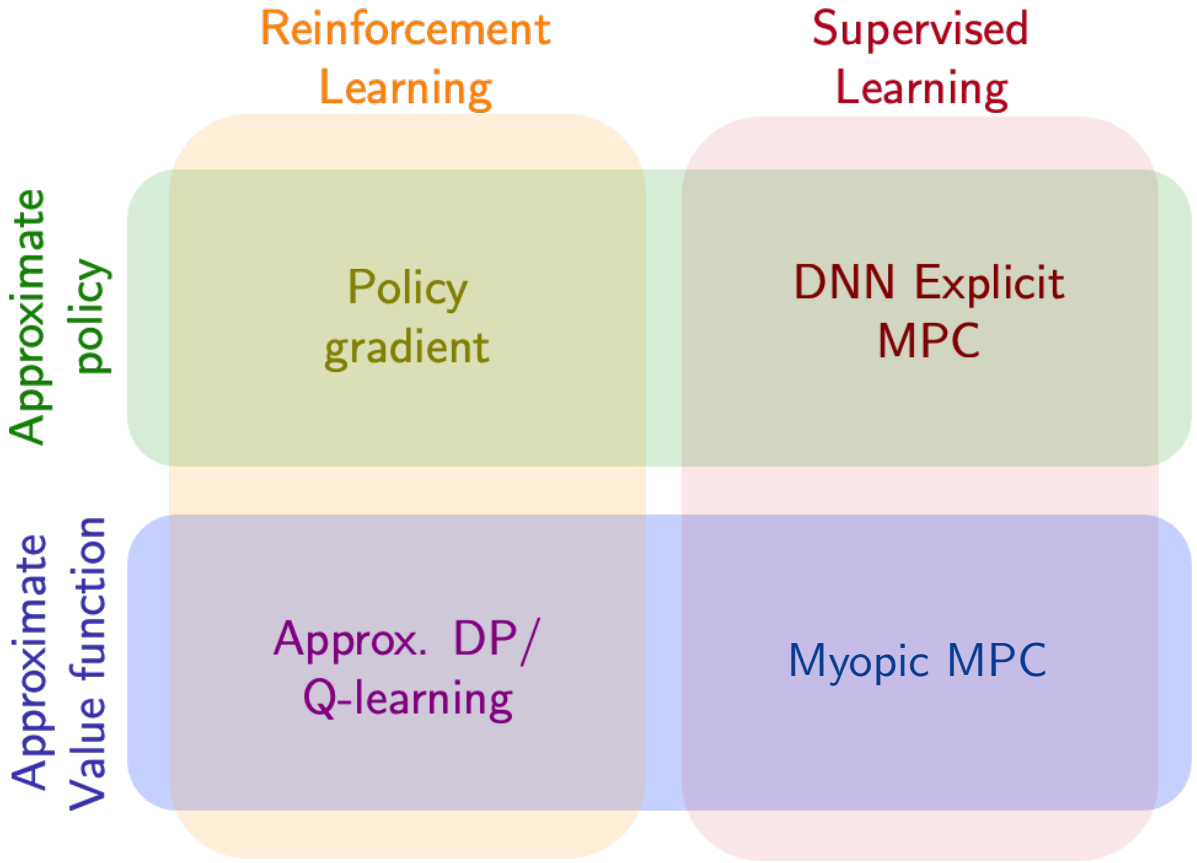




- Tightened constraints
- Sampling time: $3 \rightarrow 0.1$s

Value function approximation facilitates flexibility

Concluding Remarks



- Accurate representation of policy crucial
- No changes to controller parameters

- Requires high fidelity simulators or real system
- Several episodes to learn